UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA

PROYECTO FINAL

ANÁLISIS DE SISTEMAS 2

VIVENT

Sistema SaaS multiempresa de gestión de inventarios

Documentación del proyecto: factibilidades, requerimientos, diagramas UML, modelo entidad/relación, diseño arquitectónico, diseño de componentes, diseño de interfaz y experiencia de usuario, evaluación del diseño y refactorización aplicada.

INTEGRANTES

Christian Méndez

Wilfero Delgado

Fernando Palencia

Guatemala, octubre de 2026

Índice

1. Factibilidades

1.1 Factibilidad operativa · 1.2 Factibilidad técnica · 1.3 Factibilidad financiera · 1.4 Resumen

2. Levantamiento de requerimientos

2.1 Requerimientos funcionales · 2.2 Requerimientos no funcionales · 2.3 Clases (modelo conceptual) · 2.4 Relaciones (cardinalidades) · 2.5 Campos (diccionario resumido) · 2.6 Reglas de negocio que condicionan el modelo

3. Diagramas UML

Casos de uso (9): login, compra, venta POS, anulación de venta, ajuste de inventario, traslado, cierre de caja, reportes, administración · Secuencias (6): login, compra, venta POS, traslado, ajuste, anulación · Actividades (4): venta POS, traslado, compra, cierre

4. Diagrama Entidad/Relación

4.1 E/R (A) estructura del tenant, seguridad y catálogo · 4.2 E/R (B) inventario, transacciones y auditoría · 4.3 Interpretación · 4.4 Vista completa (anexo)

5. Diseño arquitectónico

5.1 Arquitectura elegida y justificación · 5.2 Vistas arquitectónicas (lógica, procesos, desarrollo, física)

6. Diseño de componentes

6.1 Diagrama de componentes · 6.2 Cohesión y acoplamiento · 6.3 Diseño basado en clases · 6.4 Realización a nivel de componentes

7. Diseño de la interfaz y de la experiencia de usuario

7.1 Elementos de UX aplicados · 7.2 Diseño de la interfaz (UI) · 7.3 Usabilidad y accesibilidad · 7.4 Diseño para la movilidad

8. Evaluación del diseño

9. Refactorización de componentes aplicada

Anexo A — Código fuente PlantUML

1. Factibilidades del proyecto Vivent

Vivent es un SaaS de inventarios multiempresa (multi-tenant por subdominio) para empresas pequeñas. El estado actual del repositorio ya cuenta con el núcleo del MVP implementado y probado, lo que cambia la naturaleza de este análisis: las factibilidades se evalúan sobre un sistema real en funcionamiento, no sobre una idea.

1.1 Factibilidad operativa

Pregunta: ¿hay equipo y recursos para operar el sistema?

Equipo y desarrollo

Rol	Descripción
Desarrollo	3 integrantes: Christian Méndez, Wilfero Delgado y Fernando Palencia
Pruebas	Suite de integración sobre PostgreSQL real, ejecutada en Docker
Capacidad probada	Alcance del curso completado: los 6 módulos y los 3 procesos integrados

Operación del producto

Aspecto	Descripción
Onboarding de empresas	Alta por seed SQL o aprovisionamiento manual desde el portal master
Usuarios piloto	2 a 5 compañías pequeñas, 2 a 3 usuarios por compañía, 1 sucursal inicial
Capacitación requerida	Mínima: la interfaz está traducida (es-GT / en-US) y los permisos ocultan módulos no autorizados
Operación diaria	El personal de cada empresa confirma pagos de transferencia, atiende pedidos digitales y usa el POS interno
Soporte	Vía portal master (bitácora de auditoría propia, reinicio de contraseñas, gestión de planes y suscripciones)

Conclusión operativa

Factible. El equipo actual (tres integrantes) ya opera el ciclo completo. La operación de las empresas piloto requiere personal con capacitación básica de punto de venta e inventario, que es el perfil que estas empresas ya emplean. El riesgo operativo principal es la confirmación manual de pagos (transferencias), que exige disciplina interna del personal autorizado — mitigado con auditoría y estados automáticos de expiración.

1.2 Factibilidad técnica

Pregunta: ¿se cuenta con el personal capacitado y la tecnología adecuadas?

Stack tecnológico ya validado en ejecución real

Componente	Tecnología
Backend	Java 25 LTS, Spring Boot 4.0.6, Spring Security + JWT, Spring Data JPA, Flyway, PostgreSQL 16 + PostGIS
Base de datos	Esquema creado desde cero con migraciones versionadas: 34 migraciones + 21 seeds idempotentes verificadas en bootstrap limpio
Frontend	Next.js 16.2.6, React 19, TypeScript, Tailwind CSS v4, TanStack Query/Table, React Hook Form + Zod, next-intl
Storefront	Módulo de comercio digital dentro del frontend Next.js (`vivent-front/src/features/storefront`); la landing de marketing es un sitio estático Astro independiente (`nabrix-gt/vivent-landing`)
Despliegue	Docker Compose (PostGIS, backend, frontend, nginx)
Pruebas	Suite de integración sobre PostgreSQL real en Docker
Calidad	ESLint + TypeScript estricto; build de 68 rutas frontend

Capacidades técnicas ya demostradas

- Aislamiento multi-tenant autoritativo en backend (JWT + `company_id`), no solo en frontend.
- Inventario en unidad base con conversiones por presentación y costeo FIFO/PEPS por capas.
- Kardex completo: todo movimiento de inventario queda registrado.
- Reservas de inventario identificables y expirables con workers seguros (`FOR UPDATE SKIP LOCKED`).
- Auditoría y bitácora en toda acción sensible.
- Procesamiento confiable (inbox/outbox, idempotencia) reutilizable para integraciones externas.

Dependencias externas

Dependencia	Estado
PostgreSQL + PostGIS	Instalado y operativo localmente (Docker)
Photon (geocoding)	Servicio público gratuito; con caché y fallback manual
Google Maps JavaScript API	Opcional; hay fallback con coordenadas manuales
Microservicio de correo (Nabrix)	Contrato implementado; pendiente de activar con API key
Meta Cloud API (WhatsApp)	Diseñado (runtime y flujos listos); integración real planificada como Fase 6
Pasarela de pagos	Fuera de alcance actual (cobro manual en el MVP)

Conclusión técnica

Factible. El stack es estándar, gratuito en licencias de desarrollo, y ya corre end-to-end con pruebas automatizadas. La única pieza de integración mayor pendiente (Meta/WhatsApp) está deliberadamente

encapsulada detrás del motor conversacional ya construido y no bloquea el núcleo.

1.3 Factibilidad financiera

Pregunta: ¿el dinero va a alcanzar?

Costos de infraestructura (estimación mensual en GTQ)

Concepto	Costo estimado	Nota
VPS / hosting backend (4 GB RAM)	Q250 – Q400	Docker Compose: PostgreSQL, backend, frontend, nginx
Landing de marketing (Astro estático)	Q0	Hosting gratuito suficiente para la escala piloto
Dominio (corpoviva.com)	~Q12/año	Solo anual
Certificados TLS	Q0	Let's Encrypt
Almacenamiento de imágenes	Q0 – Q50	Disco del propio servidor (fase inicial)
Google Maps API	Q0	No contratado; se usa Photon y coordenadas manuales
Meta Cloud API (WhatsApp)	Variable	Se cobra por conversación según categoría; solo empresas que contraten el Complemento lo asumen (se traspasa como costo del servicio)
Total fijo mensual	**≈ Q250 – Q450**	Para 2–5 empresas

Ingresos proyectados (modelo de monetización ya definido)

Vivent monetiza por planes de suscripción + Complementos (add-ons) con cobro de instalación más mensualidad en GTQ, registrado ya en base de datos:

Concepto	Precio (según seed actual)
Plan TRIAL	Gratuito (30 días)
Planes BASIC / PLUS / PRO	Precios parametrizables por la plataforma
WEB_STOREFRONT (tienda web)	Q1,500 instalación + Q250/mes
WHATSAPP_COMMERCE	Por definir (COMING_SOON)
CUSTOM_DOMAIN	Por definir (COMING_SOON)
ONLINE_PAYMENTS	Por definir (COMING_SOON)

Escenario ilustrativo de equilibrio (ejemplo, no compromiso)

```
5 empresas × plan BASIC Q250/mes        = Q1,250/mes
2 empresas × WEB_STOREFRONT Q250/mes    = Q500/mes   (+ instalación ya cobrada)
3 empresas × WHATSAPP_COMMERCE Q300/mes = Q900/mes   (precio supuesto)
-----------------------------------------
Ingreso mensual recurrente estimado     ≈ Q2,650/mes
Costo fijo mensual                      ≈ Q450/mes
Margen antes de desarrollo              ≈ Q2,200/mes
```

El costo de desarrollo está amortizado: el sistema ya está construido. El costo marginal de incorporar una empresa nueva al mismo VPS es cercano a cero (filas en base de datos + seeds).

Conclusión financiera

Factible. El costo fijo de infraestructura es bajo y el modelo de ingresos (suscripción + complementos con instalación) recupera la inversión inicial con pocas empresas. Los únicos costos variables relevantes (WhatsApp por conversación, pasarela de pagos cuando se contrate) se asocian a Complementos que se cobran aparte, con precio que la plataforma puede ajustar explícitamente.

1.4 Resumen

Factibilidad	Veredicto	Riesgo principal
Operativa	Factible	Confirmación manual de pagos depende de disciplina del personal
Técnica	Factible	Integración Meta/WhatsApp pendiente de fase 6 (encapsulada)
Financiera	Factible	Precios de planes aún por fijar por la plataforma

2. Levantamiento de requerimientos

Fuente de verdad: reglas de negocio de `AGENTS.md` y esquema real implementado en las migraciones `V001–V018` de `vivent-back/database/migrations/`.

2.1 Requerimientos funcionales (resumen)

Código	Requerimiento
RF-01	Plataforma multiempresa: cada compañía aislada por `company_id`, identificada por subdominio
RF-02	Autenticación por subdomain + username + password con JWT y permisos
RF-03	Roles como plantillas de permisos + overrides GRANT/DENY por usuario
RF-04	Catálogo de productos con unidad base y N presentaciones con factor de conversión
RF-05	Compras por cualquier presentación; al confirmar crean capas FIFO y kardex
RF-06	Compra confirmada no editable; corrección por anulación con motivo
RF-07	Ventas tipo POS por cualquier presentación; validación de stock disponible
RF-08	Costeo FIFO/PEPS: consumo de capas más antiguas primero
RF-09	Kardex: todo movimiento de inventario registrado con costo y balance
RF-10	Anulación de venta con permiso, motivo obligatorio y disposición de productos
RF-11	Traslados entre sucursales: solicitud → aprobación (reserva) → despacho → recepción parcial o total
RF-12	Ajustes de inventario por motivo (daño, pérdida, vencimiento, conteo…) con aprobación
RF-13	Cierre de caja por usuario/sucursal/fecha con diferencia y aprobación
RF-14	Reportes básicos (existencias, stock bajo, kardex, ventas, compras, traslados, ajustes)
RF-15	Auditoría de toda acción sensible
RF-16	Suscripciones y planes con límites (sucursales, usuarios, productos)
RF-17	Eliminación siempre lógica (soft delete)
RF-18	Sin factura legal en el MVP; impuesto informativo parametrizable

2.2 Requerimientos no funcionales

Código	Requerimiento
RNF-01	El backend es autoritativo: permisos, tenant, stock y estados se validan en el servidor
RNF-02	Aislamiento estricto por compañía; nada cruza `company_id`
RNF-03	Trazabilidad: auditoría, kardex y estados históricos; prioridad sobre simplicidad
RNF-04	Concurrencia: bloqueos pesimistas y `FOR UPDATE SKIP LOCKED` en operaciones sensibles
RNF-05	Código interno en inglés; traducción solo desde la interfaz (es-GT / en-US)
RNF-06	Esquema reproducible desde cero con migraciones versionadas (Flyway, UTC, out-of-order)
RNF-07	Seguridad de credenciales: BCrypt, tokens opacos hasheados, secretos nunca en logs
RNF-08	Personalización por empresa: logo, colores, tema, idioma, moneda, zona horaria
RNF-09	Paginación en listados y validación de entrada en toda API

2.3 Clases (modelo conceptual)

Las clases del modelo coinciden con las entidades del esquema (diseño dirigido por base de datos). Responsabilidades principales:

Grupo: estructura de la plataforma

Clase	Responsabilidad	Tabla
Company	Empresa/tenant: identidad, branding, moneda, impuesto, zona horaria	`companies`
Branch	Sucursal operativa de una compañía	`branches`
Location	Ubicación física dentro de la sucursal (bodega, mostrador, dañados…)	`locations`
Plan	Plantilla de límites y módulos habilitados	`plans`
Subscription	Estado comercial de una compañía (TRIAL, ACTIVE…)	`subscriptions`

Grupo: identidad y seguridad

Clase	Responsabilidad	Tabla
User	Empleado de una compañía con sucursal y rol activos	`users`
Role	Plantilla de permisos (sistema o de compañía)	`roles`
Permission	Permiso atómico `module.action`	`permissions`
RolePermission	Asociación rol ↔ permiso	`role_permissions`
UserPermissionOverride	Concesión/denegación especial por usuario	`user_permission_overrides`
UserBranchAssignment	Asignación (histórica) de usuario a sucursal	`user_branch_assignments`
AuditLog	Bitácora de acciones sensibles	`audit_logs`

Grupo: catálogo

Clase	Responsabilidad	Tabla
Category	Categoría con jerarquía opcional	`categories`
Product	Producto maestro con unidad base y mínimo de stock	`products`
ProductUnit	Catálogo de unidades (can, kg, liter…)	`product_units`
ProductPresentation	Presentación vendible: barcode, factor, precios	`product_presentations`
Supplier	Proveedor de la compañía	`suppliers`
Customer	Cliente; incluye consumidor final genérico	`customers`

Grupo: inventario

Clase	Responsabilidad	Tabla
InventoryBalance	Existencia por sucursal/ubicación/producto; available generado	`inventory_balances`
InventoryLayer	Capa FIFO: cantidad recibida/restante, costo, lote, vencimiento	`inventory_layers`
StockMovement	Kardex: entrada/salida con costo y balance posterior	`stock_movements`
InventoryAdjustment	Ajuste con tipo, dirección y flujo de aprobación	`inventory_adjustments`
InventoryAdjustmentLine	Línea de ajuste	`inventory_adjustment_lines`

Grupo: transacciones

Clase	Responsabilidad	Tabla
Purchase	Compra a proveedor con flujo DRAFT → CONFIRMED → (VOIDED)	`purchases`
PurchaseDetail	Línea de compra por presentación (conversión a unidad base)	`purchase_details`
Sale	Venta POS con flujo DRAFT → CONFIRMED → (CANCELLED)	`sales`
SaleDetail	Línea de venta con costo FIFO aplicado	`sale_details`
Transfer	Traslado entre sucursales con estados de flujo	`transfers`
TransferDetail	Línea solicitada/aprobada/recibida en unidad base	`transfer_details`
CashClosure	Cierre de caja diario por usuario	`cash_closures`
CashClosurePayment	Desglose del cierre por método de pago	`cash_closure_payments`

2.4 Relaciones (cardinalidades reales)

Relación	Cardinalidad	Nota
Company – Branch	1 a N	`branch.company_id` obligatorio; sin "sucursal maestra"
Branch – Location	1 a N	Tipo MAIN/SALES_FLOOR/DAMAGED/IN_TRANSIT/CUSTOM; `is_sellable`
Company – Subscription	1 a N	1 suscripción vigente por compañía
Plan – Subscription	1 a N	Límites de sucursales, usuarios y productos
Company – User	1 a N	Username único dentro de la compañía (`company_id, username`)
Branch – User	1 a N	Usuario pertenece a una sucursal activa
Role – User	1 a N	Rol plantilla; roles de sistema globales (`company_id` nulo)
Role – Permission (RolePermission)	N a M	Los roles agregan permisos
User – Permission (Override)	N a M	GRANT después de DENY en la evaluación
User – Branch (Assignment)	N a M histórico	Con `is_primary` y vigencia
Company – Category	1 a N	Jerarquía opcional (`parent_category_id`)
Category – Product	1 a 0..N	Categoría opcional en producto
ProductUnit – Product	1 a N	Unidad base por código (can, kg…)
Product – Presentation	1 a N	Factor > 0; barcode por presentación
Company – Supplier / Customer	1 a N	Nivel compañía, no sucursal
(Branch, Location, Product) – Balance	N a 1	Único por combinación; disponible generado
Purchase – PurchaseDetail	1 a N	Detalle referencia producto + presentación
PurchaseDetail – InventoryLayer	1 a 0..N	La capa conserva el origen de la compra
Purchase – Supplier	N a 1	Compra apunta a una sucursal de ingreso
Sale – SaleDetail	1 a N	Líneas con `fifo_cost_total`
Sale – Customer	N a 0..1	Consumidor final = cliente genérico o nulo
CashClosure – CashClosurePayment	1 a N	Un registro por método de pago
Transfer – TransferDetail	1 a N	Solicitado / aprobado / recibido en base
Adjustment – AdjustmentLine	1 a N	Cantidad en unidad base obligatoria

Entidades de negocio – StockMovement	1 a N polimórfico	`source_type/source_id` apuntan a compra, venta, traslado, ajuste…
Entidades de negocio – AuditLog	1 a N polimórfico	`entity_type/entity_id` + acción

2.5 Campos (diccionario resumido)

Detalles completos en las migraciones; aquí los campos clave por entidad con sus reglas de validación:

companies

```
id uuid PK · legal_name(150) · commercial_name(150)
tenant_slug(80) UNIQUE patrón ^[a-z0-9-]+$ · domain(255) UNIQUE
primary_color(7)=#2563eb · secondary_color(7)=#64748b · default_theme=light|dark
supported_locales text[] no vacío · default_locale=es-GT
currency_code(3)=GTQ · timezone(80)=America/Guatemala
tax_enabled bool · tax_rate numeric(5,2) 0..100
status ACTIVE|INACTIVE|SUSPENDED|DELETED · deleted_at nullable
```

branches / locations

```
branches: company_id FK · name(150) · code(50) UNIQUE(company) · is_active
locations: branch_id FK · type enum · is_default · is_active · is_sellable
```

users

```
company_id FK · branch_id FK · role_id FK · username(80) UNIQUE(company)
password_hash(255) BCrypt · first_name(100) · last_name(100) · email(150)
phone nullable · birth_date nullable
status ACTIVE|INACTIVE|LOCKED|DELETED · must_change_password bool
last_login_at · deleted_at nullable
```

roles / permissions / overrides

```
roles: company_id nullable (nulo = sistema) · code(100) · is_system
permissions: code(100) UNIQUE formato module.action · module_code · action_code
overrides: user_id FK · permission_id FK UNIQUE(user,permission) · effect GRANT|DENY
```

products / product_presentations / product_units

```
products: category_id nullable · base_unit_code FK a product_units.code
          name(150) · sku nullable · is_stockable · has_expiration_date|lot|serial
          minimum_stock numeric >= 0 · deleted_at nullable
presentations: name UNIQUE(product) · barcode(60) nullable
          conversion_factor numeric(19,6) > 0 (permite decimales, ej. 0.5)
          purchase_price_suggested >= 0 · sale_price >= 0
          is_default_purchase · is_default_sale
product_units: code PK (unit, can, bottle, bag, kg, lb, meter, liter)
          allows_decimal bool
```

inventory_balances

```
UNIQUE(company, branch, location, product)
quantity_on_hand >= 0 · quantity_reserved >= 0
quantity_available = GENERATED ALWAYS AS (on_hand - reserved) STORED
```

inventory_layers (FIFO)

```
UNIQUE scope + purchase_detail_id nullable · source_type/source_id polimórfico
quantity_received > 0 · 0 <= quantity_remaining <= received
unit_cost >= 0 · expiration_date · lot_number
status OPEN|CONSUMED|VOIDED
```

stock_movements (kardex)

```
movement_type: PURCHASE_IN, SALE_OUT, SALE_CANCEL_IN, PURCHASE_VOID_OUT,
  TRANSFER_RESERVED, TRANSFER_OUT, TRANSFER_IN, ADJUSTMENT_IN, ADJUSTMENT_OUT,
  STOCK_COUNT_IN/OUT, LOSS_OUT, DAMAGE_OUT, EXPIRED_OUT
CHECK: exactamente una dirección (in>0 y out=0, o viceversa)
unit_cost / total_cost / balance_after nullable · notes · created_by
```

purchases / purchase_details

```
purchases: purchase_number UNIQUE(company) · supplier_invoice_number nullable
  purchase_date date · payment_method CASH|CARD|TRANSFER|CHECK|OTHER
  payment_type CASH|CREDIT · subtotal/tax/discount/total >= 0
  status DRAFT|PENDING_APPROVAL|CONFIRMED|VOIDED
  created_by · confirmed_by/at · voided_by/at · void_reason
details: presentation FK · quantity_presentation > 0 · conversion_factor > 0
  quantity_base > 0 · unit_cost_presentation / unit_cost_base >= 0
  expiration_date · lot_number
```

sales / sale_details

```
sales: sale_number UNIQUE(company) · sale_date · customer_id nullable
  payment_method CASH|CARD|TRANSFER|CREDIT|OTHER
  subtotal/discount/tax/total >= 0
  status DRAFT|CONFIRMED|CANCELLED · confirmed_at
  cancelled_by/at · cancel_reason ·
  cancel_disposition RETURN_TO_STOCK|DAMAGED|LOSS|DO_NOT_RETURN
details: barcode · quantity_presentation/base · unit_price >= 0
  discount_amount · tax_amount · total_amount · fifo_cost_total >= 0
```

transfers / transfer_details

```
transfers: transfer_number UNIQUE(company)
  source_branch_id <> destination_branch_id (CHECK)
  status REQUESTED|APPROVED|REJECTED|IN_TRANSIT|PARTIALLY_RECEIVED|RECEIVED|CANCELLED
  requested_by · approved_by/at · dispatched_by/at · received_by/at
  cancelled_by/at · cancel_reason · notes
details: quantity_requested_presentation/base · quantity_approved_base
  quantity_received_base (recepción parcial con evidencia del faltante)
```

inventory_adjustments / lines

```
adjustments: adjustment_number UNIQUE(company) · adjustment_type
  (PHYSICAL_COUNT, DAMAGE, LOSS, THEFT, EXPIRED, INTERNAL_USE,
   REGISTER_ERROR, SUPPLIER_RETURN, CUSTOM)
  direction IN|OUT · status DRAFT|PENDING_APPROVAL|APPROVED|REJECTED|CANCELLED
  approved_by/at · rejected_by/at · rejection_reason
lines: product FK · presentation nullable · quantity_base > 0
  unit_cost nullable · notes
```

cash_closures / cash_closure_payments

```
closures: UNIQUE(company, branch, user, closure_date)
  confirmed_sales_total · cancelled_sales_total · discount_total
  expected_total · counted_total · difference_amount (todos >= 0)
  status OPEN|CLOSED|APPROVED|REOPENED · approved_by/at · totals_calculated_at
payments: UNIQUE(closure, payment_method) · amount >= 0
```

audit_logs

```
action: CREATE|UPDATE|DELETE|DEACTIVATE|LOGIN|LOGOUT|CONFIRM|VOID|
        CANCEL|APPROVE|REJECT|DISPATCH|RECEIVE|AUTHORIZE
entity_type(80) · entity_id · old_value/new_value jsonb
ip_address inet · user_agent · branch_id nullable
```

plans / subscriptions

```
plans: code UNIQUE · price >= 0 · max_branches/users/products > 0
       enabled_modules text[] · is_custom · is_active
subscriptions: status TRIAL|ACTIVE|EXPIRED|SUSPENDED|CANCELLED
       starts_at · ends_at · trial_ends_at
```

2.6 Reglas de negocio que condicionan el modelo

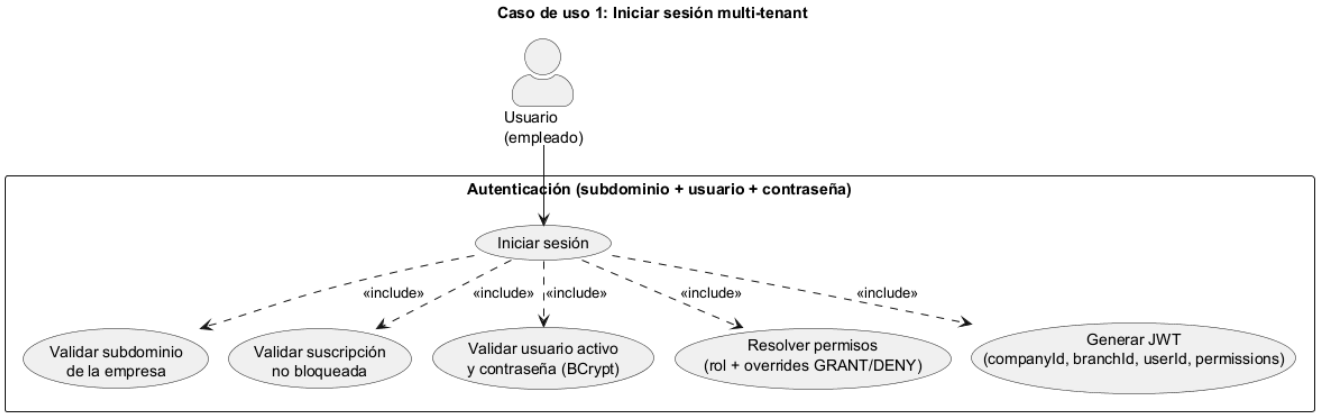
1. El inventario se controla solo en unidad base; las presentaciones son vistas vendibles con factor de conversión.
2. `quantity_available = on_hand − reserved` es columna generada: nunca se calcula mal.
3. Vender sin stock está prohibido por defecto (`INSUFFICIENT_STOCK`).
4. Compra confirmada es inmutable; venta confirmada solo se anula con permiso + motivo + disposición.
5. Los traslados reservan stock en origen desde la aprobación y lo materializan en la recepción.
6. Las eliminaciones son lógicas (`deleted_at`): nada se borra físicamente.
7. Todo movimiento → kardex; toda acción sensible → auditoría.

3. Diagramas UML

Cada diagrama se presenta como imagen renderizada; el código fuente PlantUML está disponible en los archivos `.puml` junto a cada imagen. Casos de uso pequeños, uno por flujo del sistema, como se solicitó.

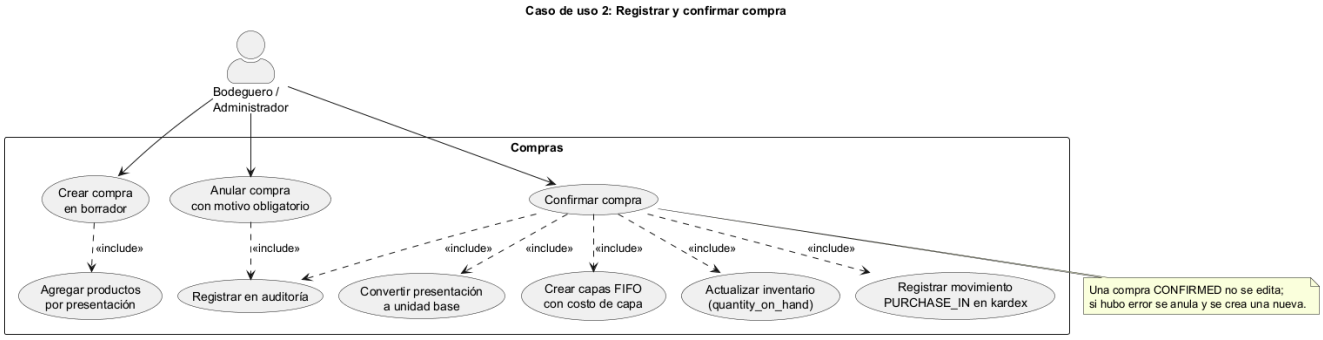
Grupo	Diagramas
Casos de uso	9 (login, compra, venta POS, anulación, ajuste, traslado, cierre, reportes, administración)
Secuencia	6 (login, compra, venta, traslado, ajuste, anulación)
Actividades	4 (venta POS, traslado, compra, cierre)

Caso de uso 1 — Iniciar sesión multi-tenant



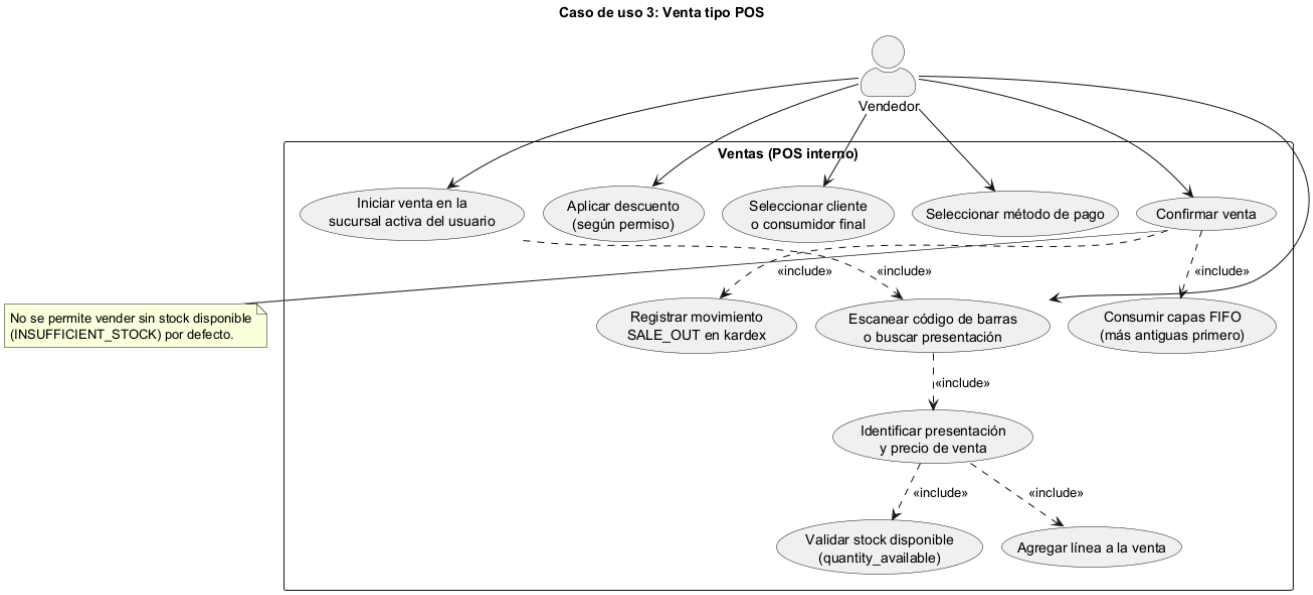
Caso de uso 1 — Iniciar sesión multi-tenant

Caso de uso 2 — Registrar y confirmar compra



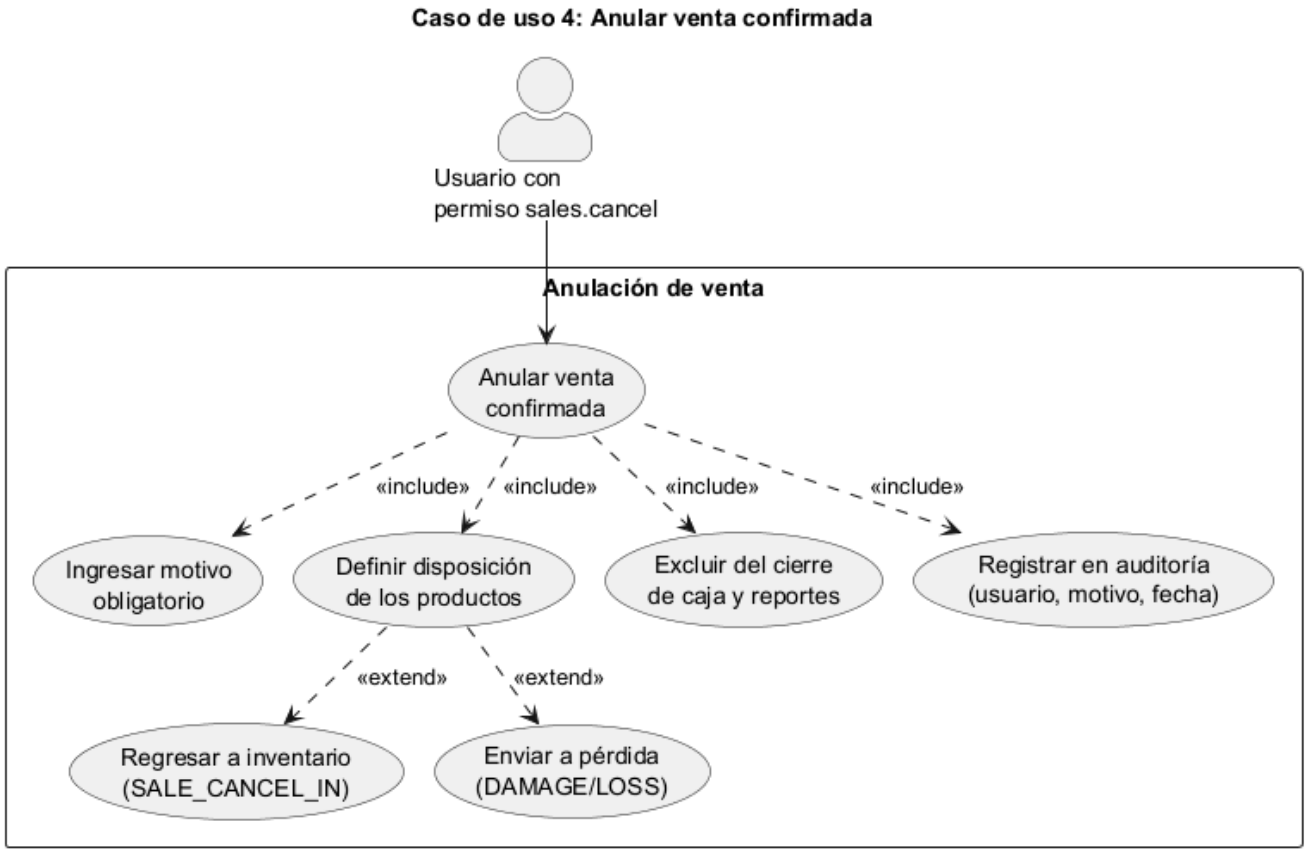
Caso de uso 2 — Registrar y confirmar compra

Caso de uso 3 — Venta tipo POS



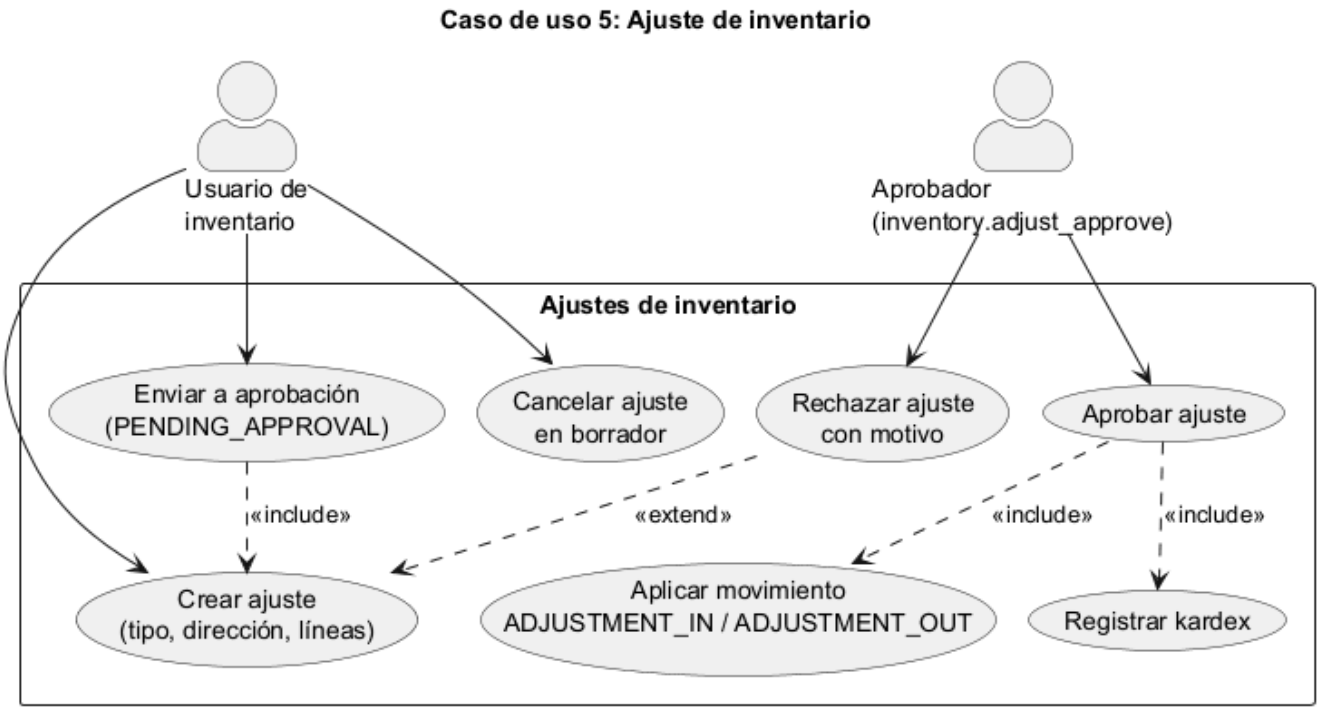
Caso de uso 3 — Venta tipo POS

Caso de uso 4 — Anular venta confirmada



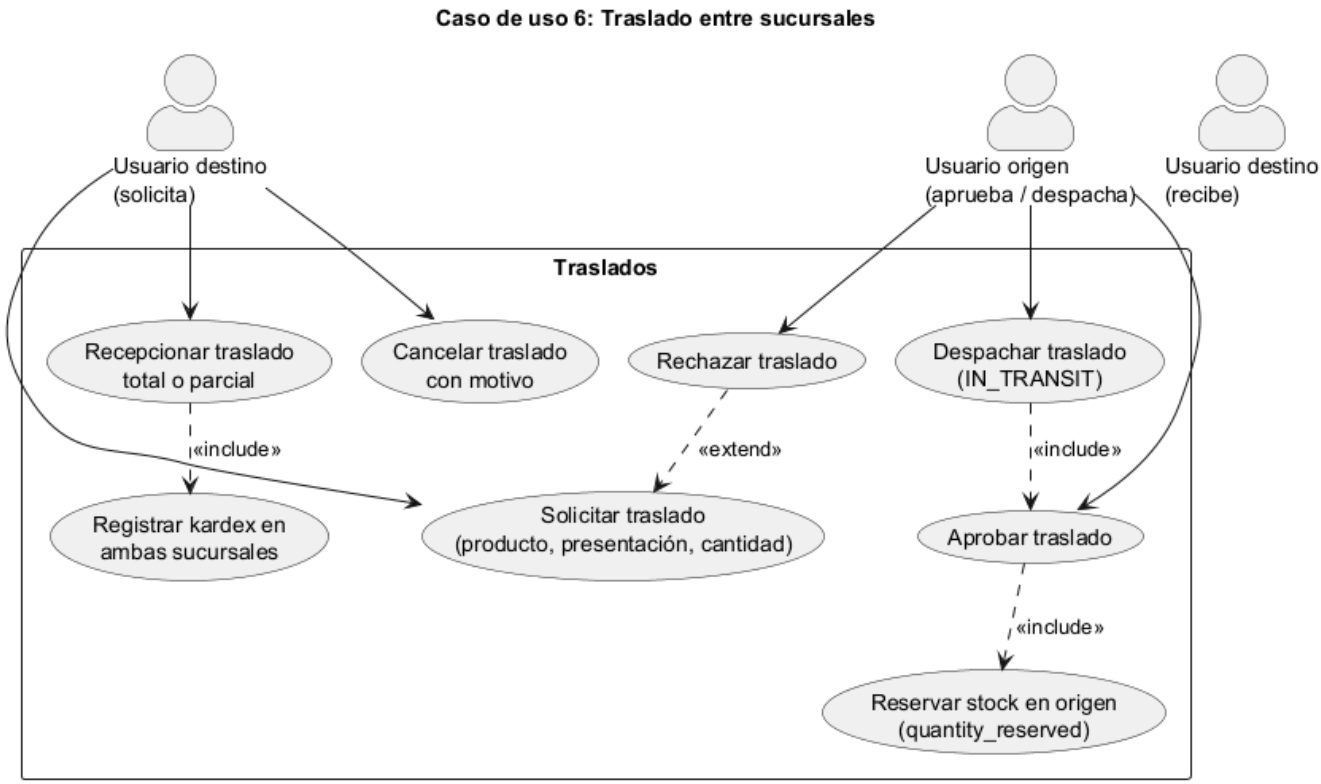
Caso de uso 4 — Anular venta confirmada

Caso de uso 5 — Ajuste de inventario



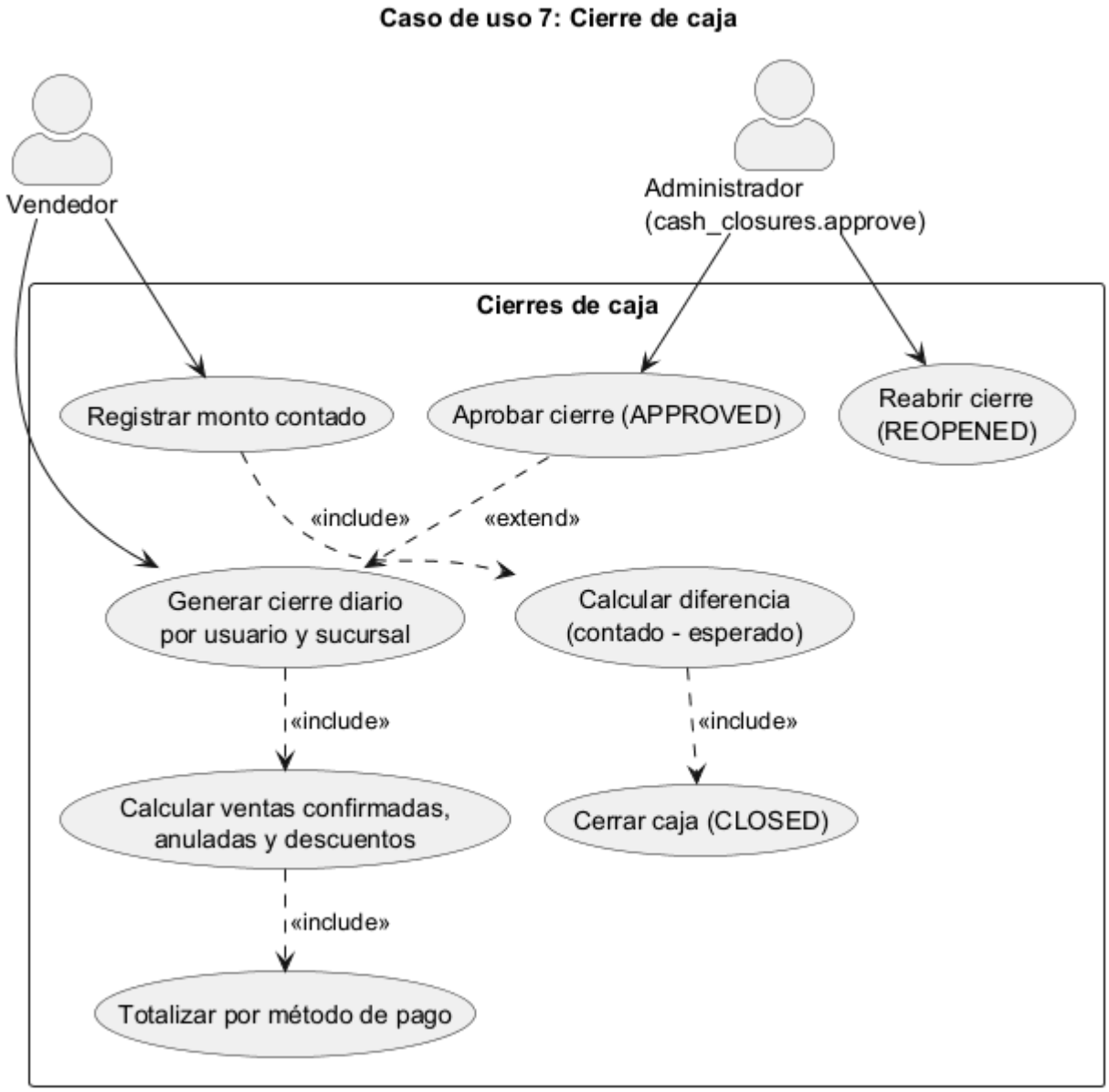
Caso de uso 5 — Ajuste de inventario

Caso de uso 6 — Traslado entre sucursales



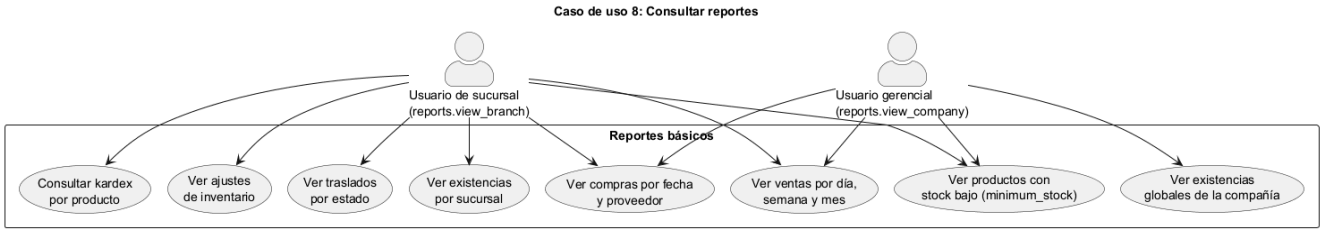
Caso de uso 6 — Traslado entre sucursales

Caso de uso 7 — Cierre de caja



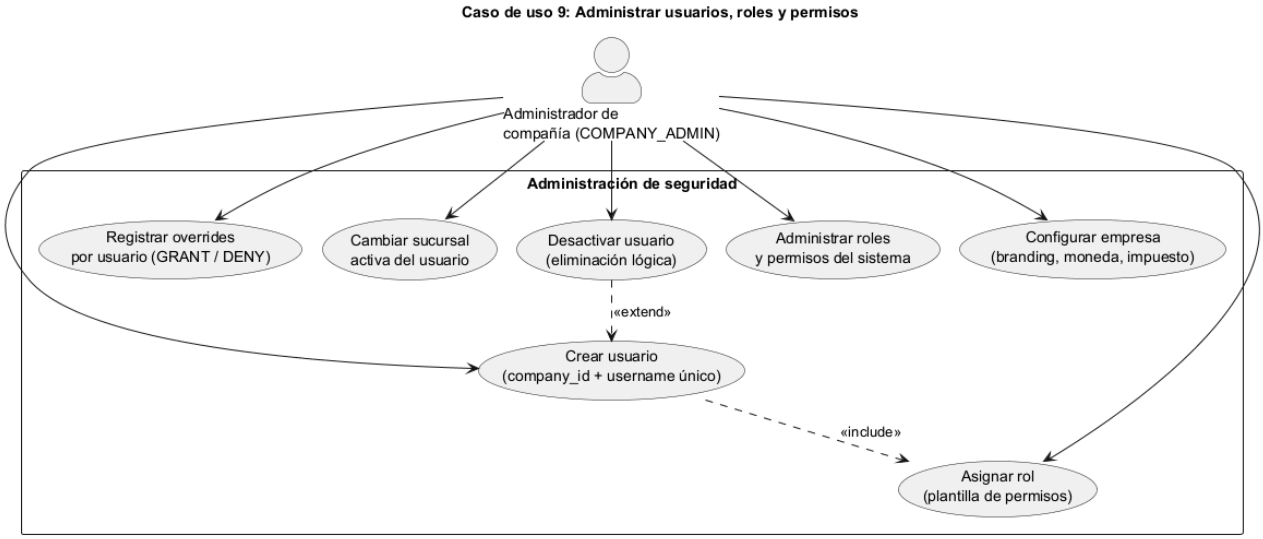
Caso de uso 7 — Cierre de caja

Caso de uso 8 — Consultar reportes



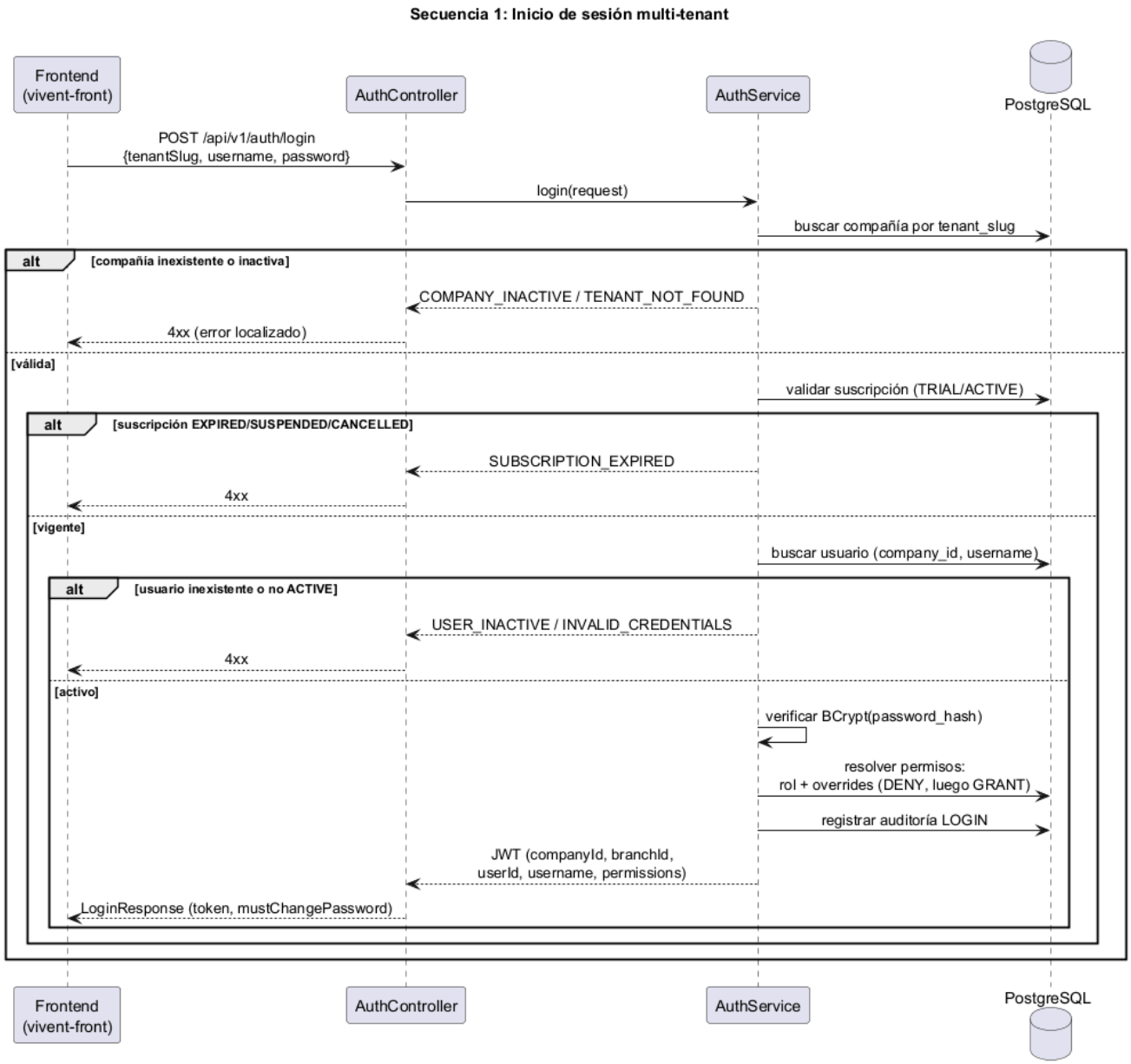
Caso de uso 8 — Consultar reportes

Caso de uso 9 — Administrar usuarios, roles y permisos



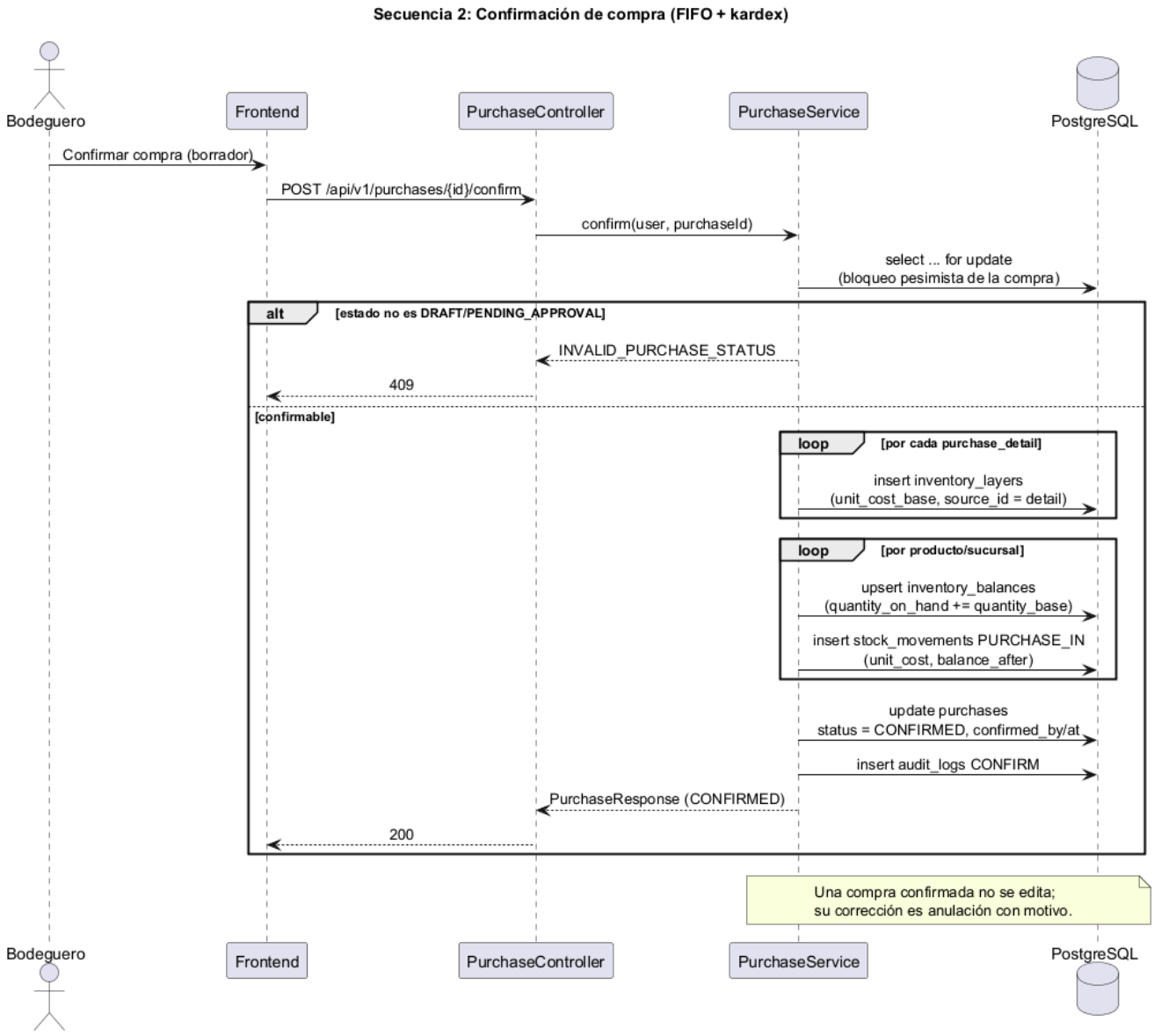
Caso de uso 9 — Administrar usuarios, roles y permisos

Secuencia 1 — Inicio de sesión multi-tenant



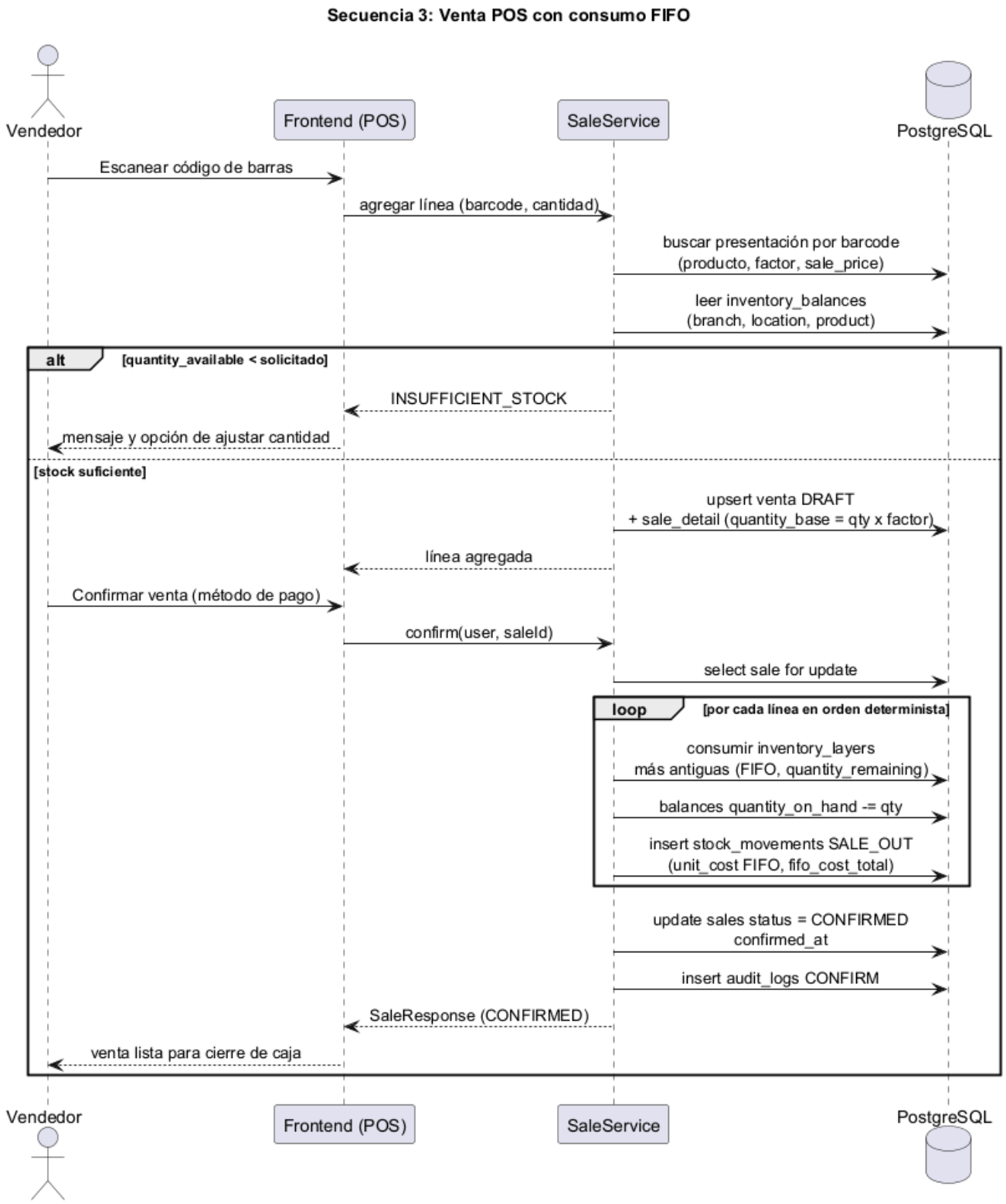
Secuencia 1 — Inicio de sesión multi-tenant

Secuencia 2 — Confirmación de compra (FIFO + kardex)



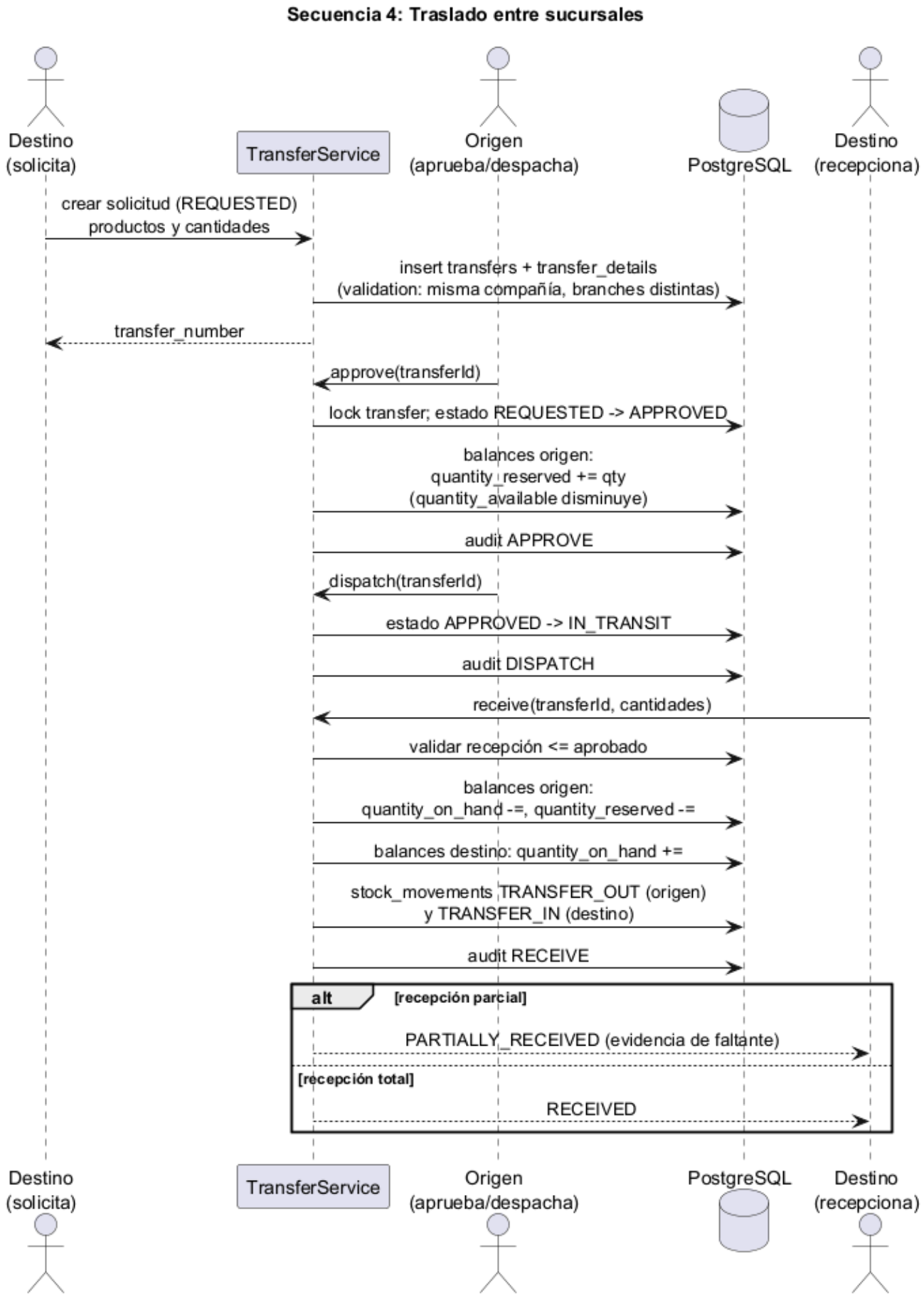
Secuencia 2 — Confirmación de compra (FIFO + kardex)

Secuencia 3 — Venta POS con consumo FIFO



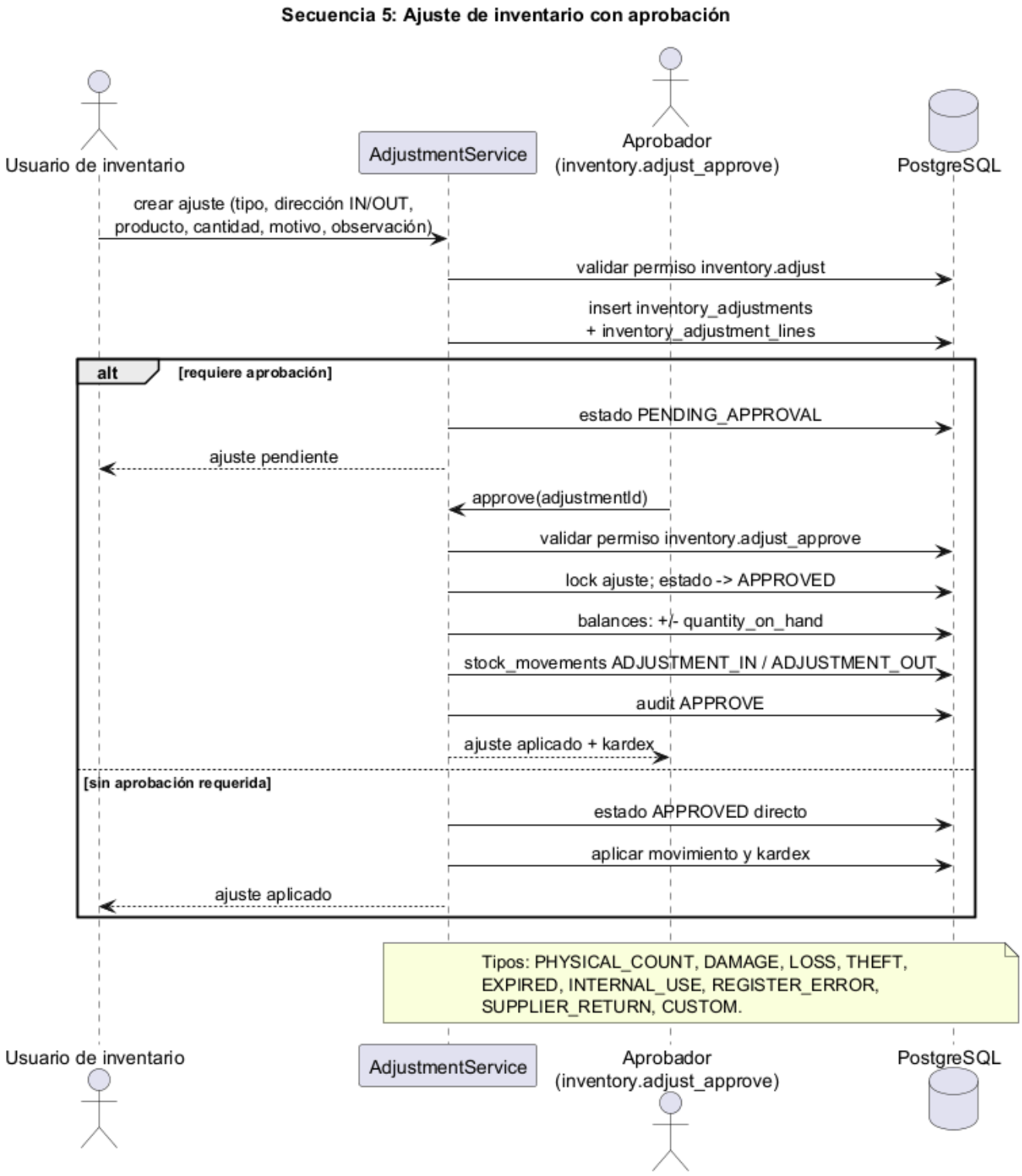
Secuencia 3 — Venta POS con consumo FIFO

Secuencia 4 — Traslado entre sucursales



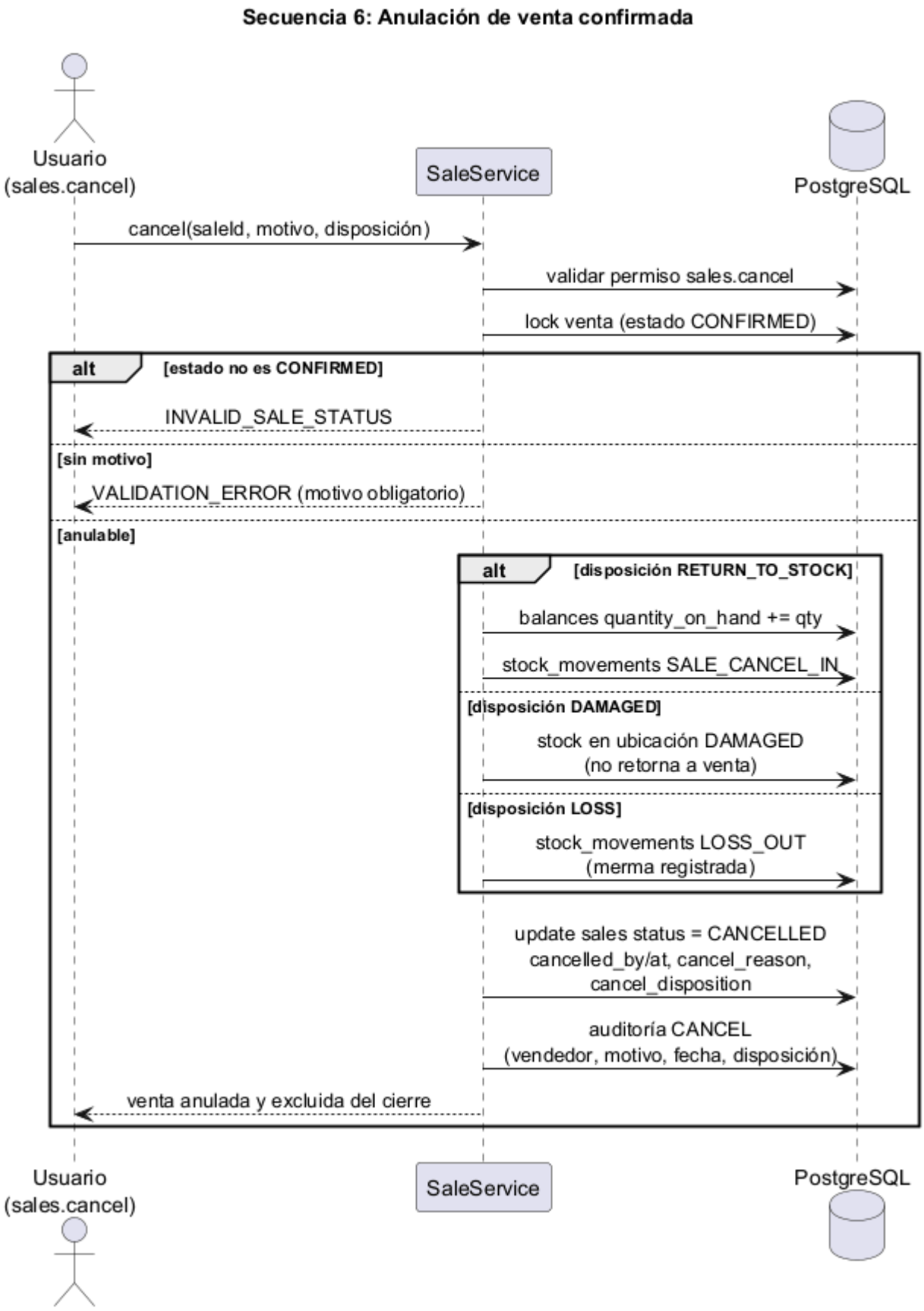
Secuencia 4 — Traslado entre sucursales

Secuencia 5 — Ajuste de inventario con aprobación



Secuencia 5 — Ajuste de inventario con aprobación

Secuencia 6 — Anulación de venta confirmada



Secuencia 6 — Anulación de venta confirmada

Actividad 1 — Venta POS



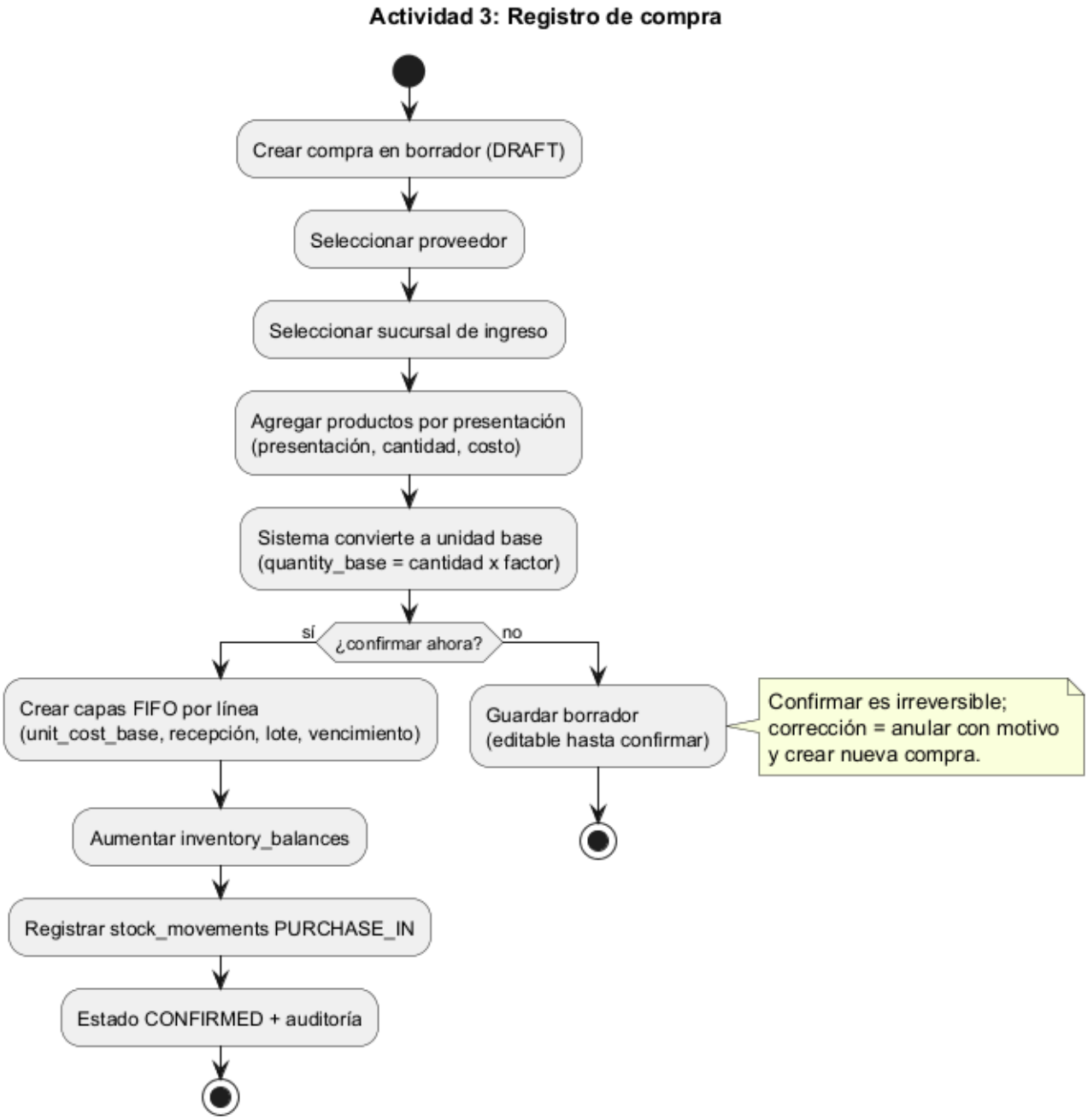
Actividad 1 — Venta POS

Actividad 2 — Traslado entre sucursales



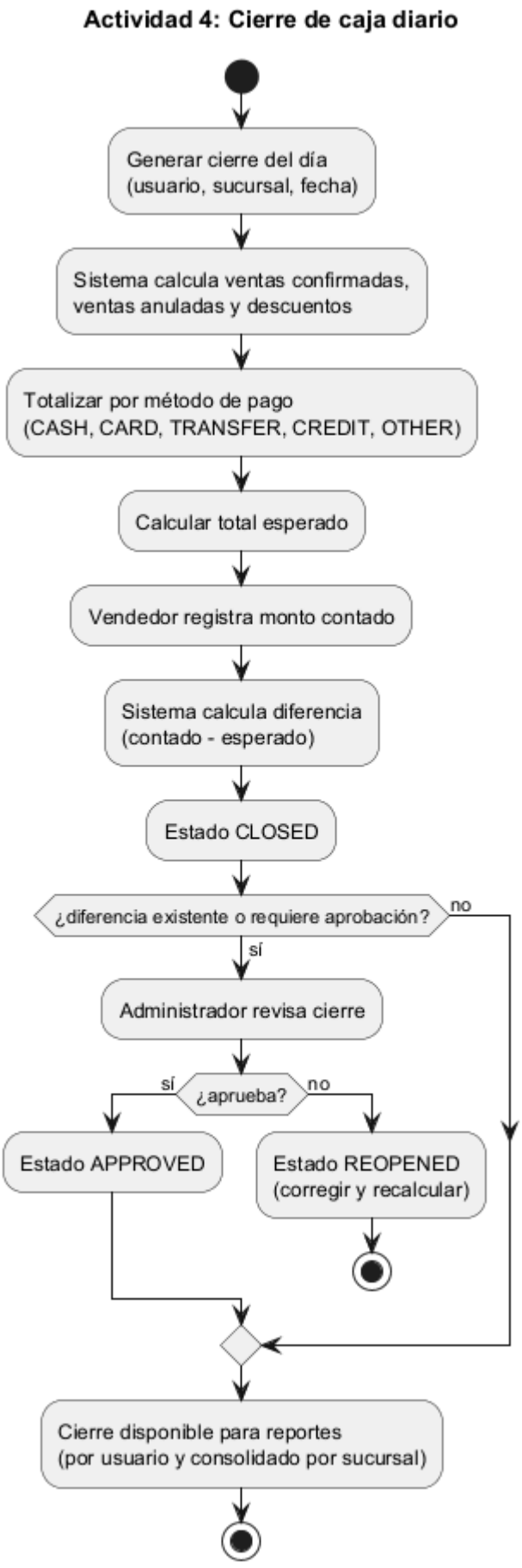
Actividad 2 — Traslado entre sucursales

Actividad 3 — Registro de compra



Actividad 3 — Registro de compra

Actividad 4 — Cierre de caja diario



Actividad 4 — Cierre de caja diario

4. Diagrama Entidad/Relación

El modelo se presenta en dos diagramas legibles: (A) estructura del tenant, seguridad y catálogo; (B) inventario, transacciones y auditoría. Al final se conserva la vista completa como anexo.

4.1 Diagrama E/R (A) — Estructura del tenant, seguridad y catálogo

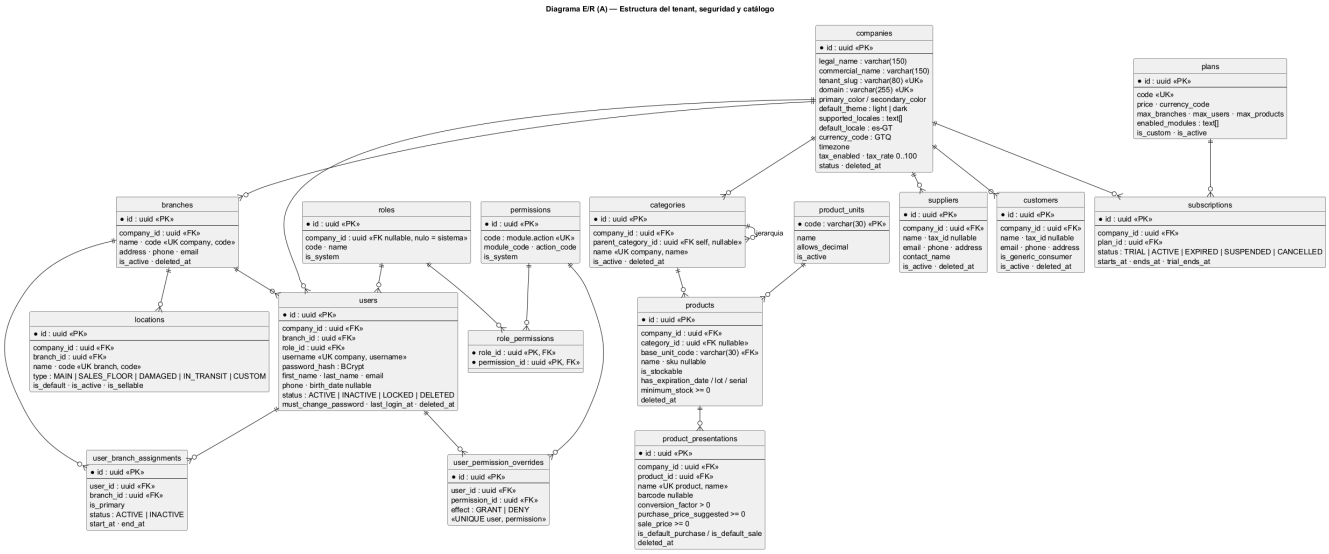


Diagrama E/R (A) — Estructura del tenant, seguridad y catálogo

4.2 Diagrama E/R (B) — Inventario, transacciones y auditoría

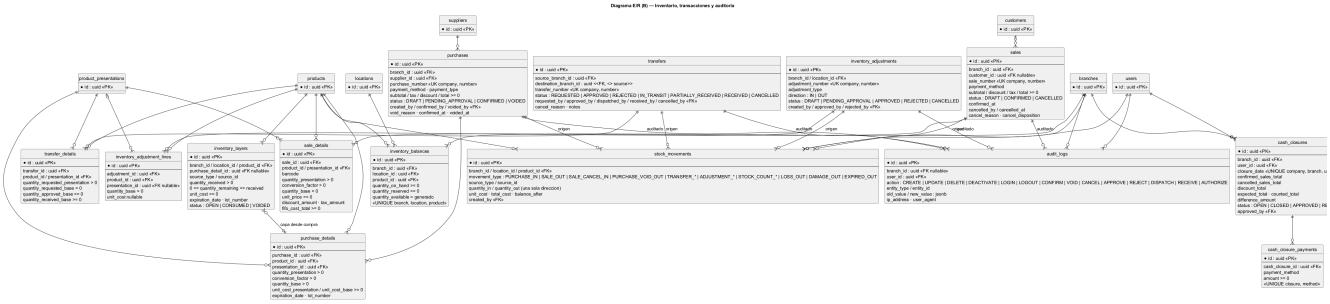


Diagrama E/R (B) — Inventario, transacciones y auditoría

4.3 Interpretación

El diagrama refleja el esquema real de PostgreSQL implementado en las migraciones `V001–V018` de `vivent-back/database/migrations/`. Decisiones de diseño visibles en el modelo:

1. Estructura jerárquica del tenant

```
companies → branches → locations
```

- `companies` es la raíz del aislamiento: toda entidad operativa lleva `company_id`.
- No existe "sucursal maestra": los reportes globales se calculan desde `company_id`.
- `locations` clasifica espacios físicos (MAIN, SALES_FLOOR, DAMAGED, IN_TRANSIT, CUSTOM) con `is_sellable` que define dónde puede venderse/consumirse stock.

2. Seguridad y permisos

```
roles → role_permissions ← permissions
users → user_permission_overrides → permissions
```

- Los roles son plantillas (`is_system` para globales, `company_id` para roles propios).
- Los overrides por usuario se evalúan: permisos base → DENY → GRANT.
- `user_branch_assignments` conserva el historial de asignaciones a sucursales con vigencia.

3. Catálogo con conversiones

```
product_units → products → product_presentations
```

- El producto declara su unidad base (`base_unit_code`).
- Cada presentación declara `conversion_factor` (cuántas unidades base representa; admite decimales, p. ej. media unidad).
- El código de barras vive en la presentación: es la llave de escaneo del POS.

4. Inventario: balance + capas FIFO + kardex

```
inventory_balances  → existencia por (branch, location, product)
inventory_layers    → costo FIFO/PEPS por capa de entrada
stock_movements     → kardex: todo movimiento con costo y balance
```

- `quantity_available` es columna generada (on_hand − reserved): imposible que quede desincronizada.
- Las capas FIFO nacen de `purchase_details` (o ajustes de entrada) y se consumen por antigüedad.
- `stock_movements` usa `source_type/source_id` polimórfico para apuntar a la operación que lo generó (compra, venta, traslado, ajuste…).

5. Documentos comerciales con estados

```
purchases:  DRAFT → PENDING_APPROVAL → CONFIRMED → (VOIDED)
sales:      DRAFT → CONFIRMED → (CANCELLED con disposición)
transfers:  REQUESTED → APPROVED → IN_TRANSIT → (PARTIALLY_)RECEIVED
adjustments: DRAFT → PENDING_APPROVAL → APPROVED | REJECTED
closures:   OPEN → CLOSED → APPROVED (o REOPENED)
```

Todos llevan quién (FK a `users`), cuándo (timestamps) y motivo cuando aplica. Las claves de negocio son únicas por compañía (`UNIQUE (company_id, numero)`), no globales: cada empresa numera independientemente.

6. Cierre de caja

`cash_closures` es único por (company, branch, user, date) y guarda el desglose por método de pago en `cash_closure_payments` . Los totales (confirmed_sales_total, cancelled_sales_total, discount_total, expected_total, counted_total, difference_amount) se recalculan desde `sales` y pueden reabrirse (REOPENED).

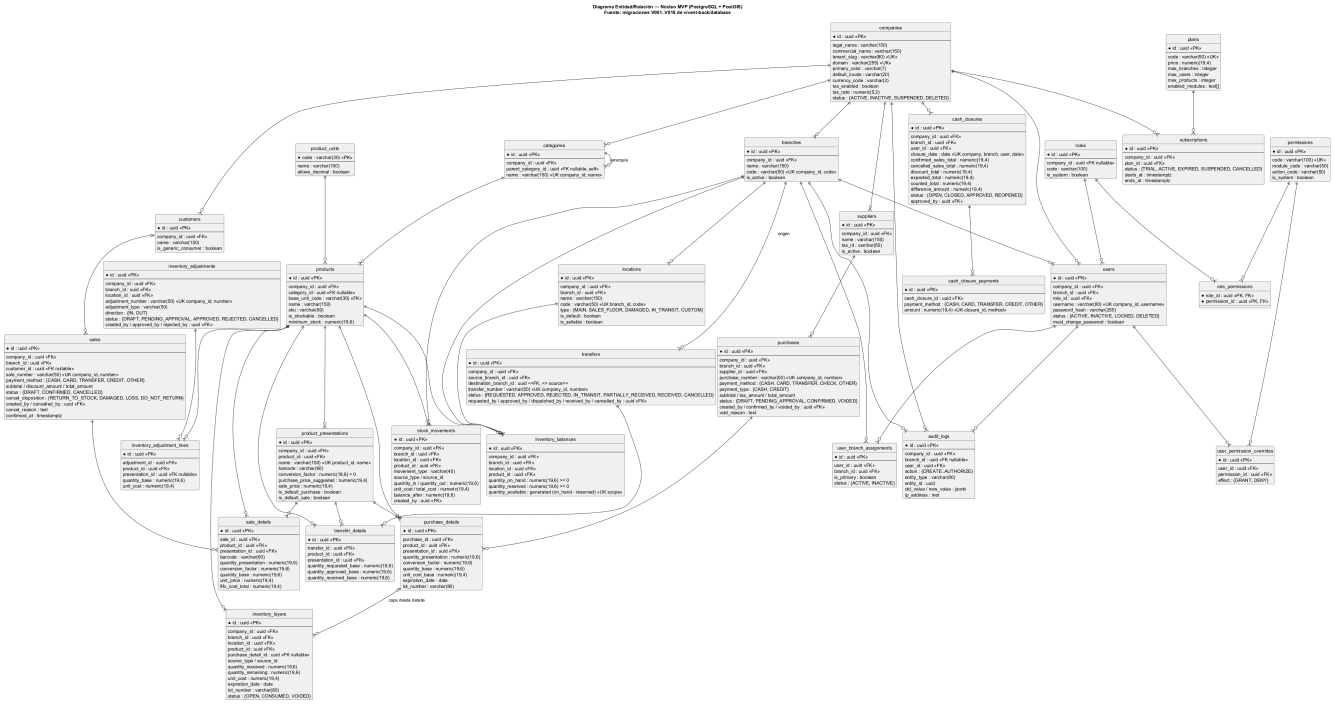
7. Auditoría

`audit_logs` registra acción, entidad afectada, valores antes/después (JSONB), IP y user agent de toda acción sensible, con usuario y sucursal de contexto.

Reglas de integridad destacadas

Regla	Implementación
Stock nunca negativo	`CHECK quantity_on_hand >= 0`
Capa consumida nunca mayor a recibida	`CHECK 0 <= quantity_remaining <= received`
Movimiento con una sola dirección	`CHECK (in>0 AND out=0) OR (out>0 AND in=0)`
Traslado entre sucursales distintas	`CHECK source_branch_id <> destination_branch_id`
Username único por compañía	`UNIQUE (company_id, username)`
Factor de conversión positivo	`CHECK conversion_factor > 0`
Eliminación lógica	`deleted_at nullable en catálogo y maestros`

4.4 Vista completa (anexo)



Vista completa del modelo Entidad/Relación (anexo)

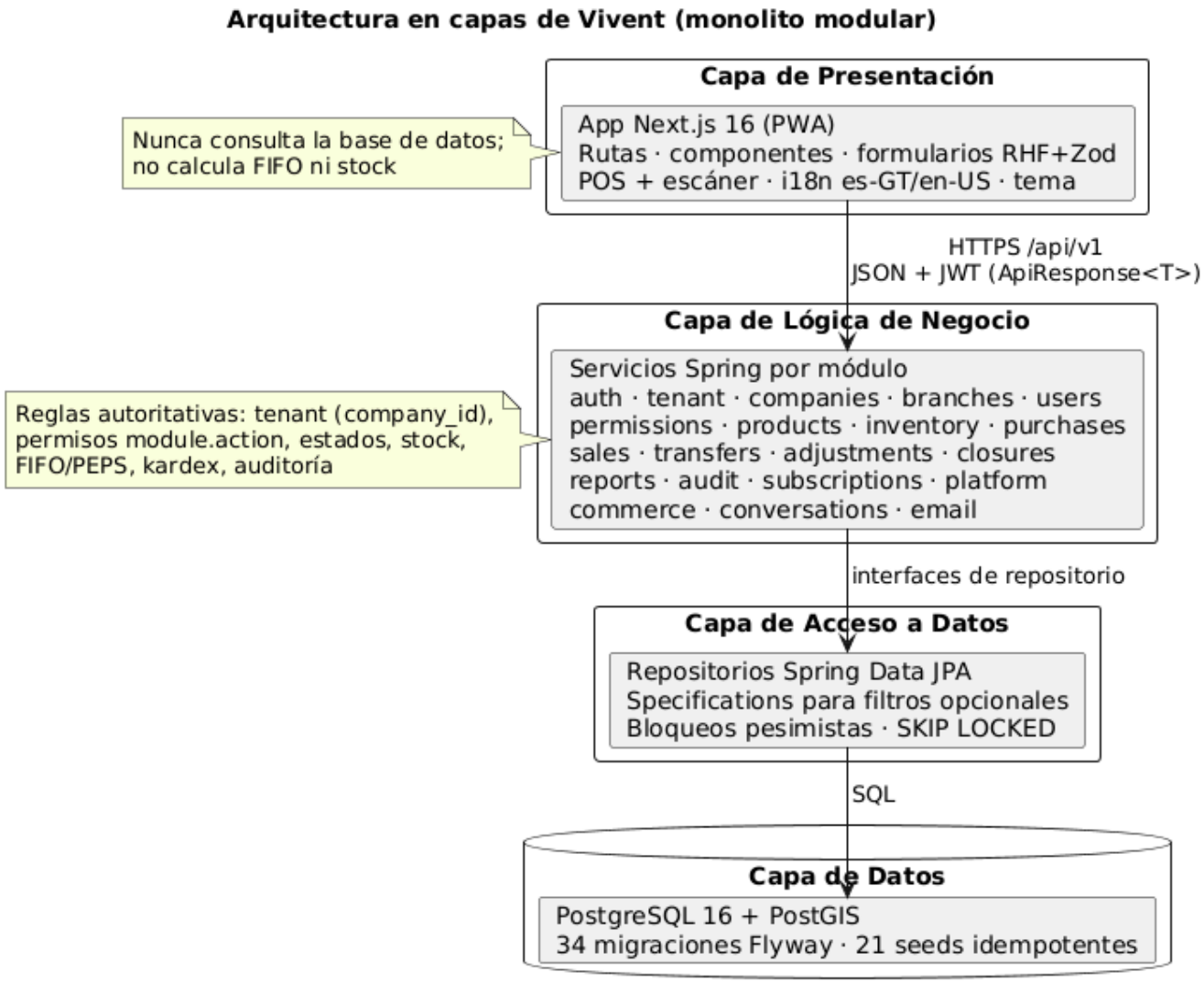
5. Diseño arquitectónico

Vivent es un **SaaS de inventarios multiempresa** cuyo diseño arquitectónico se define antes de detallar componentes. La arquitectura es el plano del sistema: organiza los módulos, las interfaces de comunicación y las tecnologías.

5.1 Arquitectura elegida

Vivent adopta una combinación de tres estilos justificada por la escala del proyecto:

```
1. Cliente-Servidor (web de tres niveles)
   Navegador/PWA  →  HTTP/HTTPS + JSON  →  Servidor de aplicación  →  PostgreSQL
2. Monolito modular en capas (N capas)
   Presentación (Next.js) | Lógica de negocio (services Spring) |
   Acceso a datos (repositorios JPA) | Base de datos (PostgreSQL + Flyway)
3. Patrón Repositorio
   Controller → Service → Repository (Spring Data) → Base de datos
```



Arquitectura en capas de Vivent (monolito modular)

Justificación frente a alternativas

Alternativa	¿Aplica a Vivent?	Motivo
Arquitectura en Capas	✅ Elegida	Sistema empresarial mediano con mantenimiento constante: separa presentación, lógica y datos
Patrón Repositorio	✅ Elegida	Muchas consultas a BD, pruebas de integración contra PostgreSQL real, cambio de fuente de datos acotado a la capa de repositorios
Cliente-Servidor	✅ Elegida	Varios usuarios (cajeros, bodega, gerencia) conectados por red a un servidor central
MVC	⚠️ No clásico	El frontend usa arquitectura de componentes (Next.js App Router), no MVC de servidor; el backend usa capas + repositorio
Microservicios	❌ Descartado por ahora	2–5 empresas piloto, un responsable: la complejidad operativa (gateway, mensajería, despliegues por servicio) no está justificada. El monolito modular conserva la posibilidad de extraer servicios (p. ej., `nabrix-email`) cuando el volumen lo amerite

El corte modular del monolito coincide con los módulos de negocio; cada módulo expone contratos REST versionados bajo `/api/v1` y su propia persistencia lógica (tablas del dominio), lo que permite evolucionar hacia microservicios sin reescribir la lógica.

5.2 Vistas arquitectónicas

(a) Vista lógica

Organización en módulos y clases (detalle en 2.3 y diagrama de componentes en 6.1):

```
Plataforma:  Company, Branch, Location, Plan, Subscription
Identidad:   User, Role, Permission, RolePermission, UserPermissionOverride,
             UserBranchAssignment, AuditLog
Catálogo:    Category, Product, ProductUnit, ProductPresentation, Supplier, Customer
Inventario:  InventoryBalance, InventoryLayer, StockMovement,
             InventoryAdjustment(+Line)
Operación:   Purchase(+Detail), Sale(+Detail), Transfer(+Detail),
             CashClosure(+Payment)
Comercio:    Storefront, Order, Payment, Fulfillment, Conversations (dev)
```

(b) Vista de procesos

Flujos de trabajo y su comunicación (detalle en las secuencias del capítulo 3): login multi-tenant, compra→FIFO→kardex, venta POS→consumo FIFO→kardex, traslado con reserva y recepción, ajuste con aprobación, cierre de caja; y en `dev`: reconciliación de reservas de comercio con workers `FOR UPDATE SKIP LOCKED` y despacho de correos vía outbox.

(c) Vista de desarrollo

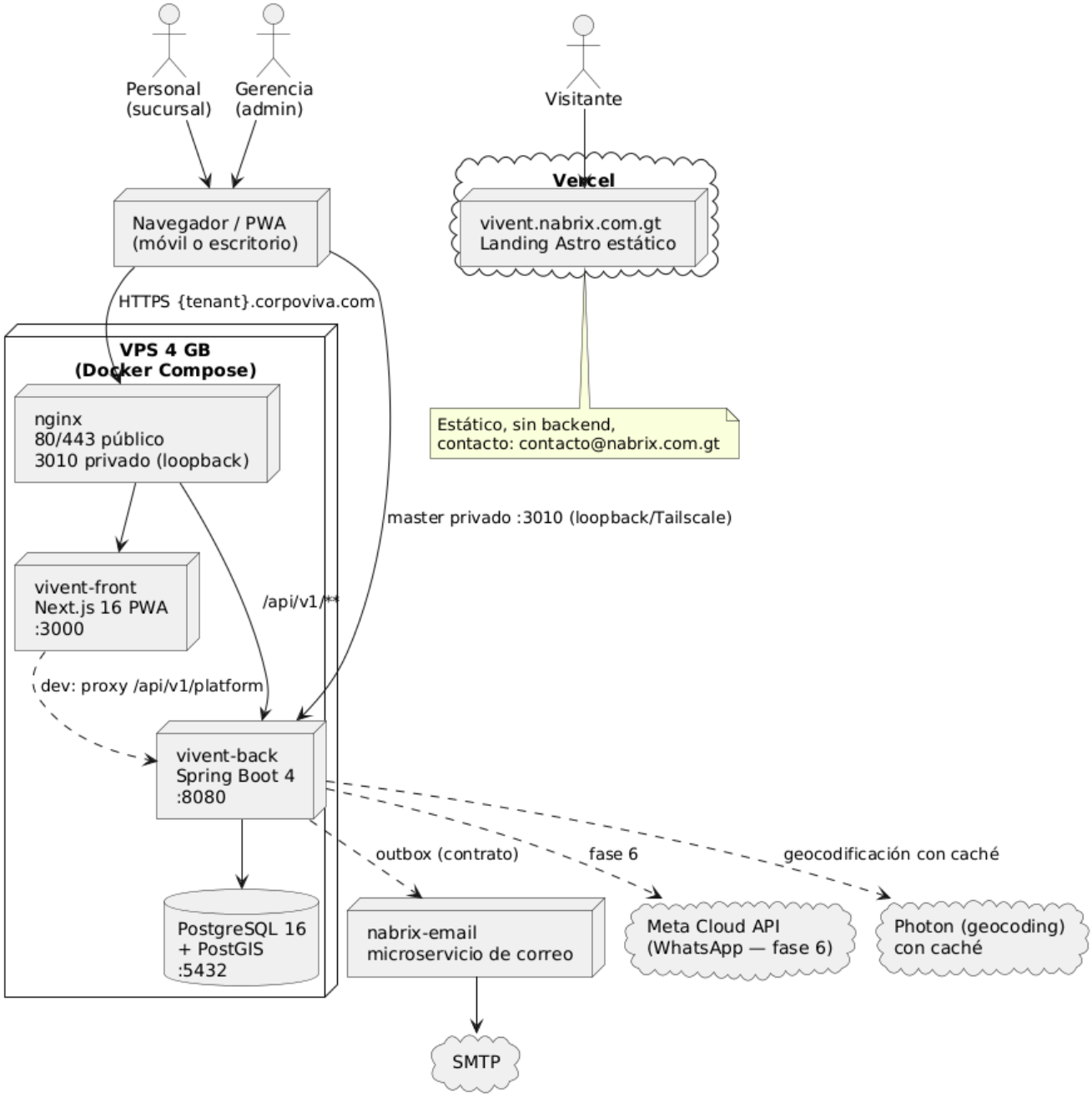
Organización real del código para trabajo modular:

```
backend/src/main/java/com/corpoviva/vivent/
  auth/ tenant/ companies/ branches/ users/ permissions/
  products/ inventory/ purchases/ sales/ transfers/ adjustments/
  closures/ reports/ audit/ subscriptions/
  commerce/{catalog,order,payment,fulfillment,aftercare}/
  conversations/{runtime,commerce}/ email/ platform/ common/ security/
  Cada módulo: controller · service · repository · entity · dto · mapper

frontend/src/
  app/           (rutas App Router por módulo)
  components/    (layout, ui, providers)
  features/      (auth, account, tenant, admin, inventory, purchases,
                  sales, transfers, adjustments, audit-logs, reports,
                  combos, platform, storefront)
  lib/           (api/http, api/errors, i18n, permissions)
  messages/      (es-GT.json, en-US.json)
```

(d) Vista física (despliegue)

```
Navegadores y PWA (cajero/móvil) → nginx (TLS, proxy, bloqueo de /master público)
  → contenedor backend Spring Boot (8080) → contenedor PostgreSQL 16 + PostGIS
  → contenedor frontend Next.js (3000)
Landing de marketing: sitio estático Astro publicado en vivent.nabrix.com.gt
Servicios externos: nabrix-email (microservicio de correo, contrato listo,
                    pendiente de API key); Meta Cloud API WhatsApp (fase 6)
```



Vista física — despliegue contenedorizado y servicios externos

Cada tenant se sirve por subdominio (`demo.corpoviva.com`), resuelto por `GET /api/v1/public/tenant-config?subdomain=` .

6. Diseño de componentes

Un componente es una unidad modular con responsabilidades relacionadas, alta cohesión y bajo acoplamiento, que se comunica por interfaces bien definidas.

6.1 Diagrama de componentes

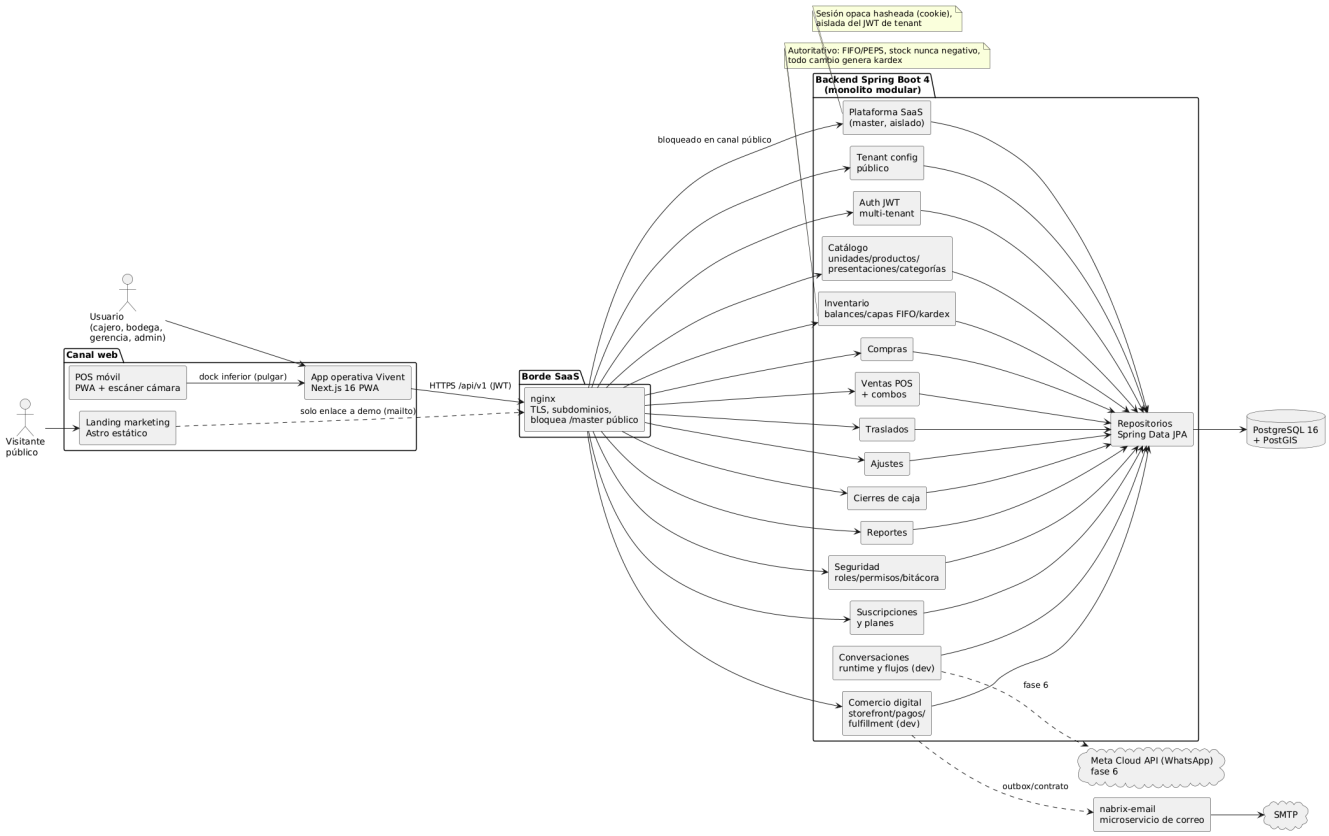


Diagrama de componentes del sistema Vivent

Componente	Responsabilidad	Interfaz expuesta	Dependencias
App operativa (Next.js PWA)	Presentación web/PWA: pantallas, formularios, POS	`REST /api/v1 (cliente HTTP src/lib/api/http.ts)`	nginx → API
Landing (Astro estático)	Marketing público, SEO, contacto	`Páginas estáticas + mailto:`	Ninguna (sin backend)
nginx	TLS, enrutamiento por subdominio, bloqueo del panel master en el canal público	`HTTP/HTTPS`	App, API
Tenant config	Configuración pública por subdominio	`GET /api/v1/public/tenant-config`	Repositorios
Auth JWT	Login subdomain+username+password, tokens y permisos	`POST /api/v1/auth/login · GET /api/v1/auth/me`	Repositorios, BCrypt
Catálogo	Unidades, categorías, productos, presentaciones	`/api/v1/product-* · /api/v1/categories`	Repositorios
Inventario	Balances, capas FIFO, kardex, stock manual	`/api/v1/inventory/*`	Catálogo, Compras
Compras / Ventas / Traslados / Ajustes / Cierres	Documentos operativos con estados	`/api/v1/purchases · /sales · /transfers · /inventory/adjustments · /cash-closures`	Inventario, Catálogo
Reportes	Summaries oficiales y consultas	`/api/v1/reports/*`	Lectura de todos los módulos
Seguridad (roles/permisos/audit)	Plantillas de permisos, overrides, bitácora	`/api/v1/roles · /permissions · /audit-logs`	Repositorios
Plataforma SaaS (master)	Alta de empresas, planes, reinicio de credenciales	`/api/v1/platform/** (cookie opaca, cadena aislada)`	Repositorios
Comercio digital (dev)	Storefront, pedidos, pagos, fulfillment	`/api/v1/commerce/** + pantallas /storefront`	Inventario, Email
Conversaciones (dev)	Runtime y flujos conversacionales (WhatsApp fase 6)	`Contratos internos + Meta Cloud API`	Comercio
Repositorios (Spring Data)	Único acceso autorizado a datos	`Interfaces por módulo`	PostgreSQL
nabrix-email	Envío de correos (outbox/inbox, idempotencia)	`Contrato de correo`	SMTP

6.2 Cohesión y acoplamiento

```
Cohesión alta:  cada módulo agrupa las operaciones que cambian por la misma razón
                (p. ej., Inventario: balances, capas FIFO, kardex, stock in/out).
Acoplamiento bajo: los servicios dependen de interfaces de repositorio (Spring Data),
                   no de SQL; el frontend consume contratos DTO (ApiResponse) sin
                   conocer la persistencia; el POS consume lookup por barcode sin
                   conocer capas FIFO; los correos se desacoplan por outbox.
Regla aplicada: la lógica de negocio nunca consulta la base de datos directamente
                (prohibido en controllers); el frontend nunca calcula FIFO, stock
                ni balances — el backend es autoritativo.
```

6.3 Diseño de componentes basados en clases

El mapeo clase → componente se hace por responsabilidad de negocio (no cada sustantivo es un componente):

```
Clases del dominio              → Componente que las encapsula
Company/Branch/Location/Plan/   → Plataforma (companies, branches,
Subscription                      subscriptions)
User/Role/Permission/Overrides/ → Identidad y seguridad (users, auth,
UserBranchAssignment/AuditLog     permissions, audit)
Product/Unit/Presentation/      → Catálogo (products, inventory para lectura)
Category/Supplier/Customer
InventoryBalance/Layer/         → Inventario (inventory)
StockMovement/Adjustment
Purchase/PurchaseDetail         → Compras (purchases)
Sale/SaleDetail                 → Ventas (sales)
Transfer/TransferDetail         → Traslados (transfers)
CashClosure/Payments            → Cierres (closures)
Storefront/Order/Payment/       → Comercio digital (commerce)
Fulfillment/Conversation
```

6.4 Realización del diseño a nivel de componentes

Contrato estándar por módulo (realización del diseño):

```
Controller  → solicitudes HTTP, validación de entrada (Jakarta Validation), thin
Service     → reglas de negocio (estados, stock, permisos, scope por sucursal)
Repository  → consultas (Specifications para filtros opcionales; sin lógica de negocio)
Entity      → mapeo a tablas Flyway; DTOs desacoplan el contrato externo
Respuesta   → ApiResponse { success, data, message, errorCode }
```

Ejemplo — componente Inventario (realización):

```
IInventoryApi (interfaz expuesta):
  GET /api/v1/inventory/summary | balances | layers | kardex | movements
  POST /api/v1/inventory/stock-in | stock-out
Realización:
  InventoryController → InventoryService (receiveStock, issueStock,
  voidPurchaseReceipt) → InventoryRepository / LayerRepository / MovementRepository
Reglas realizadas: stock nunca negativo (INSUFFICIENT_STOCK), capa FIFO más antigua
primero, todo cambio genera movimiento de kardex, reversión de compra apunta a las
capas originales por purchase_detail_id.
```

7. Diseño de la interfaz y de la experiencia de usuario

7.1 Elementos de UX aplicados (evidencia real del producto)

Elemento UX	Implementación real en Vivent
Usuario y necesidades	Roles con permisos: el menú lateral se filtra por permisos (app-navigation + sesión); cada rol ve solo sus módulos (cajero → POS; bodega → compras/inventario; gerencia → reportes/cierres)
Arquitectura de información	Rutas agrupadas por operación: `/purchases`, `/sales`, `/inventory`, `/transfers`, `/adjustments`, `/closures`, `/reports/*`, `/admin/{catalog,company,security,third-parties}`, `/master/*`
Navegación	Breadcrumbs (Inicio > Ventas > Nueva venta), menú lateral persistente con estado colapsado, alias de compatibilidad `/cash-closings` para marcadores existentes
Consistencia	Tabla estándar AdminTable, FormDialog, ConfirmDialog en todos los módulos; el botón principal siempre describe la acción ("Registrar venta", "Confirmar compra", "Aprobar")
Retroalimentación	Toasts (sonner) en crear/confirmar/cancelar; mensajes de error del backend localizados (formatApiErrorMessage); avisos de cierre desactualizado con número de ventas modificadas
Prevención de errores	Validación Zod + React Hook Form; efectivo contado ≥ 0 con 2 decimales; fechas futuras de cierre bloqueadas con la zona horaria de la compañía; avisos "No disponible" en stock; confirmaciones antes de acciones destructivas
Recuperación de errores	INSUFFICIENT_STOCK traducido a advertencia clara para el cajero; doble confirmación de compra/venta rechazada con mensaje controlado; bloqueo PASSWORD_CHANGE_REQUIRED redirige a /change-password
Personalización por empresa	Logo/nombre comercial, colores primario/secundario, tema claro/oscuro, moneda (GTQ), zona horaria e idioma por tenant
Multilenguaje	next-intl con diccionarios es-GT / en-US; los códigos internos (permisos, estados, tipos) se traducen por clave y nunca se muestran crudos

7.2 Diseño de la interfaz (UI)

Principios aplicados: claridad (botones que nombran la acción), simplicidad (una acción por pantalla, formularios con campos mínimos), consistencia (mismos controles en todo el sistema).

```
POS de escritorio:  escaneo por código de barras (Enter agrega), búsqueda con Ctrl+B,
                    finalizar con Ctrl+Enter, guardar borrador con Ctrl+S;
                    método de pago y número se piden solo al finalizar.
Formularios admin:  campos con etiquetas claras, fecha nativa del navegador,
                    listas desplegables de presentación con búsqueda por nombre/código.
Tablas:             búsqueda, paginación real del backend (PageResponse), filtros
                    avanzados en modal, exportación de la página visible a .xls.
```

7.3 Usabilidad y accesibilidad

Referencia: principios WCAG (perceptible, operable, comprensible, robusto).

```
Perceptible:    estados traducidos y con texto (no solo color: "Stock bajo",
                "Disponible"); contraste verificado en landing; iconos con etiquetas.
Operable:       navegación por teclado (skip link en landing, FAQ operable por teclado,
                atajos en POS: Enter, Ctrl+B, Ctrl+Enter, Ctrl+S).
Comprensible:   errores en lenguaje del usuario ("La contraseña es incorrecta"),
                nunca códigos crudos como "Error 0x000234" ni jerga interna
                (FIFO/backend).
Robusto:        HTML semántico, Radix UI para primitivas accesibles, contenido útil
                sin JavaScript en la landing, etiquetas identificables por lectores
                de pantalla.
```

La landing de marketing se verificó con Playwright (2026-10-04) en anchos 320/390/768/1024/1440 sin desbordes horizontales, skip link, FAQ operable por teclado, movimiento reducido (sin GSAP), contenido sin JavaScript, robots/sitemap/404. Pendiente: lectores de pantalla completos y Safari/Firefox.

7.4 Diseño para la movilidad

El usuario móvil opera con una sola mano y en entornos inestables; Vivent aplica los criterios del curso:

Criterio	Implementación en Vivent
Zona natural del pulgar (verde)	El escáner móvil se abre como dock inferior fijo (bottom-0, rounded-t-3xl) con cámara y cantidad lado a lado; las acciones principales (agregar, finalizar) están abajo
Zona de peligro (roja)	Acciones destructivas (anular venta, eliminar borrador) requieren confirmación con motivo y quedan fuera del flujo de un toque
Objetivos táctiles ≥ 44–48 px	Controles táctiles amplificados en móvil; el dock compacto separa cámara y cantidad
Glanceability (un vistazo)	El carrito aparece antes del escáner para ver lo último escaneado; el carrito baja automáticamente al agregar; jerarquía visual: producto → cantidad → total
Microcopias directas	"Registrar venta", "No disponible", "Confirmar pedido" — verbos de acción, sin jerga interna
Manejo de red inestable	PWA online-first: el service worker nunca almacena datos privados ni intercepta /api/*; offline.html como fallback de navegación; token en sessionStorage
Prueba de una mano	Flujo POS: escanear → agregar → finalizar, todo alcanzable con el pulgar
Prueba del sol	Tema claro/oscuro por preferencia y por empresa; colores por defecto con contraste estándar

La cámara y el POS móvil requieren contexto seguro (HTTPS); en desarrollo LAN la UI carga pero el navegador puede bloquear getUserMedia (comportamiento documentado).

8. Evaluación del diseño

Aplicando los criterios de evaluación del diseño al sistema real:

Criterio	Estado	Evidencia
Cumplimiento de requisitos	✅ Cumple	RF-01..RF-18 implementados en módulos; flujo completo Compra→Inventario→Venta→Kardex→Traslado→Ajuste→Reportes→Cierre validado por QA con usuarios demo
Usabilidad	✅ Cumple	Atajos de POS, toasts, errores localizados, menú por permisos, paginación real
Seguridad	✅ Cumple	JWT por tenant, BCrypt, permisos estrictos por endpoint, scope por sucursal en backend, panel master aislado (cookie opaca hasheada, lockout), secretos fuera de logs
Eficiencia y rendimiento	✅ Cumple	Paginación en listados, carga por lote de comboLines por página, índices (V009), concurrencia con bloqueos pesimistas y FOR UPDATE SKIP LOCKED
Mantenibilidad	✅ Cumple	Monolito modular con separación controller/service/repository, documentación viva (AGENTS.md, docs/, agent-context/), pruebas de integración
Integridad y consistencia de datos	✅ Cumple	quantity_available generada, CHECK de stock ≥ 0, capa ≤ recibida, una sola dirección por movimiento, unicidad por compañía, eliminación lógica

Herramientas usadas para evaluar: diagramas de casos de uso, clases, secuencias y E/R; matriz de requisitos RF → módulo; checklist de QA reproducible (`docs/QA_FUNCTIONAL_CHECKLIST.md`).

Los hallazgos detectados durante la evaluación fueron corregidos mediante las refactorizaciones documentadas en la sección 9.

9. Refactorización de componentes aplicada

Casos reales de refactorización del proyecto (separación de responsabilidades, extracción de métodos, eliminación de duplicados, reemplazo de condicionales y corrección de diseño):

#	Refactorización	Tipo aplicado	Resultado
1	Scope por sucursal: BranchService.listBranches filtraba solo por compañía y un vendedor veía otras sucursales	Corrección de responsabilidades + seguridad	Regla reforzada en 10+ servicios: solo COMPANY_ADMIN ve todas las sucursales; el resto queda acotado a su asignación; cross-branch → 403 PERMISSION_DENIED
2	Anulación de compra usaba el consumo FIFO genérico y podía revertir capas de otra compra	Extracción de operación especializada voidPurchaseReceipt (reversa por purchase_detail_id con bloqueo pesimista)	La anulación revierte exactamente las capas originales; prueba de regresión dedicada
3	Cancelación de venta con disposición DAMAGED/LOSS descontaba el stock dos veces	Corrección de flujo (SALE_CANCEL_IN + DAMAGE_OUT/LOSS_OUT con efecto neto correcto)	Efecto neto correcto por disposición; compatible hacia atrás con RETURN_TO_STOCK
4	Combos: fetch join sobre consulta paginada y cargas N+1	Optimización de acceso a datos	Paginación de combos primero y carga por lote de ítems/combos de la página
5	Listado master de empresas fallaba con lower(bytea) does not exist en PostgreSQL	Reemplazo de JPQL frágil por JpaSpecificationExecutor (predicados solo con filtros presentes)	Listado estable con y sin filtros
6	Reportes de traslados/ajustes mostraban cantidades 0 en filas	Mapeo de detalles/líneas en los listados	Filas con cantidades reales en reportes
7	next-themes causaba problemas de hidratación con el stack actual	Reemplazo por ThemeProvider propio	Tema claro/oscuro estable; prohibido reintroducir sin validar
8	npm audit con vulnerabilidad en Next.js (CVE) y lockfile npm	Migración a pnpm con --frozen-lockfile, override de postcss y allowlist de scripts	pnpm audit sin vulnerabilidades conocidas

Anexo A — Código fuente PlantUML

Los diagramas E/R se generan desde los archivos `.puml` de la carpeta `04-er/` con:

```
java -jar "$env:TEMP\plantuml\plantuml.jar" -Playout=smetana -charset UTF-8 documentacion\04-
er\*.puml
```

Los diagramas de arquitectura en capas, componentes y despliegue se generan
igual desde los fuentes incluidos en proyecto-final-anexos/plantuml/:

```
java -jar "$env:TEMP\plantuml\plantuml.jar" -Playout=smetana -charset UTF-8 proyecto-final-
anexos\plantuml\*.puml
```